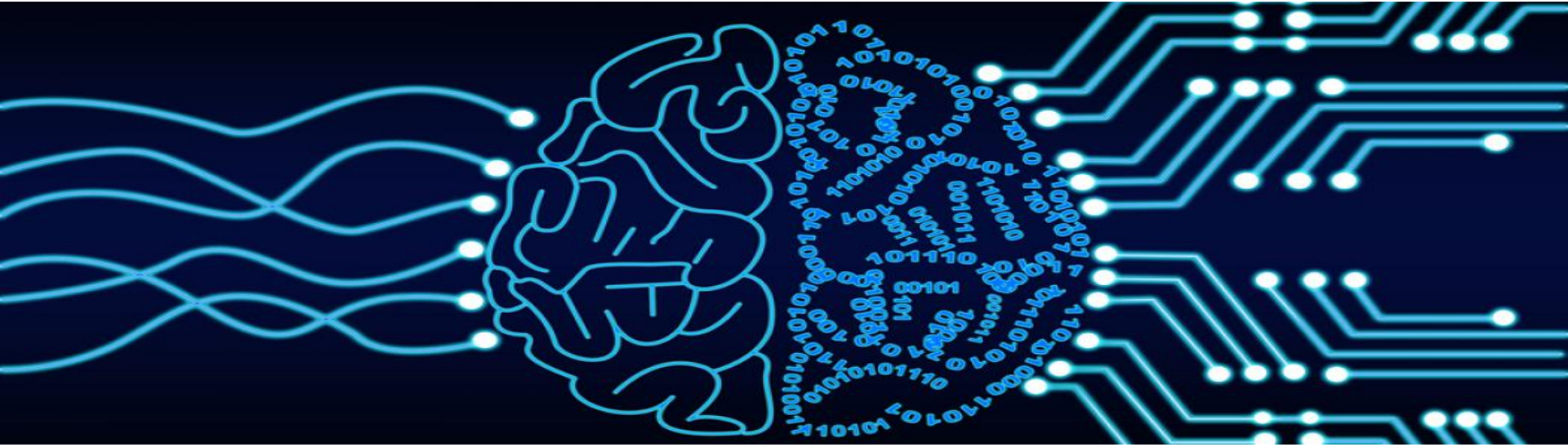


Artificial Intelligence

Chapter 3: Problem Solving Agent

Course Supervisor: Dr. Anaum Hamid

Objectives of the Chapter

- Problem Solving Agents
- Problem Formulation
- Search Strategies
- Un-informed Searching Techniques

Problem Solving Agent

Searching is one of the classic areas of AI

Problem Solving agents are one kind of goal-based agent that acts Rationally.

Problem Solving Agent

An agent that tries to come up with a sequence of actions by performing some kind of search algorithm in the background that will bring the environment into a desired state/goal.

Search

The use of search requires an abstract formulation of the problem and the available steps to construct solutions.

we need to first formulate the problem.

Problem Formulation

1. State
2. Initial State
3. Actions
4. Goal Test
5. Path Cost
6. Solution

A State Space
Set of all possible states where you can be.

A Start State. The state from where the search begins.

Actions or Successor Function

A Goal Test. A function that looks at the current state returns whether or not it is the goal state.

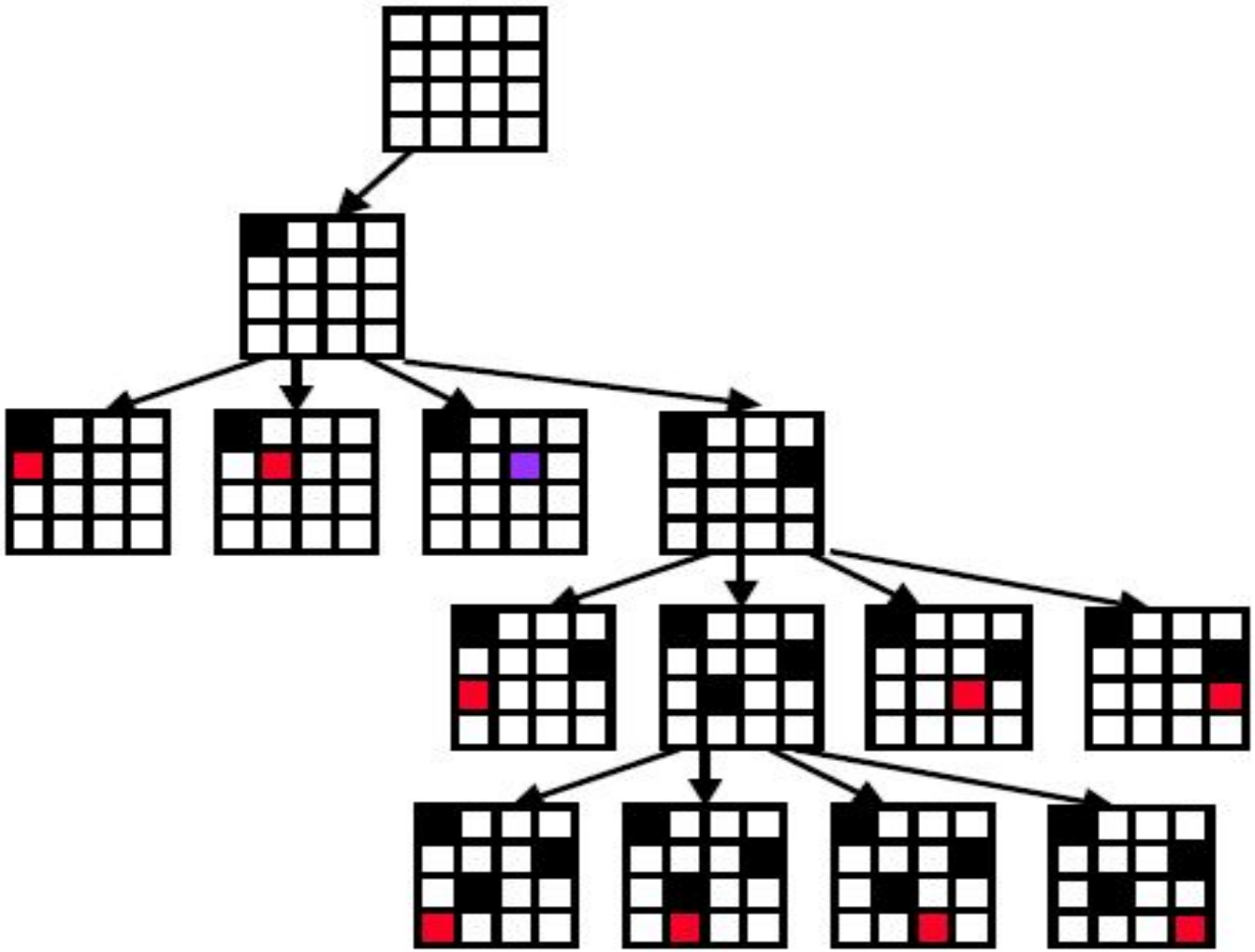
Solution to a search problem is a sequence of actions, called the **plan** that transforms the start state to the goal state.

THIS PLAN IS ACHIEVED THROUGH SEARCH ALGORITHMS

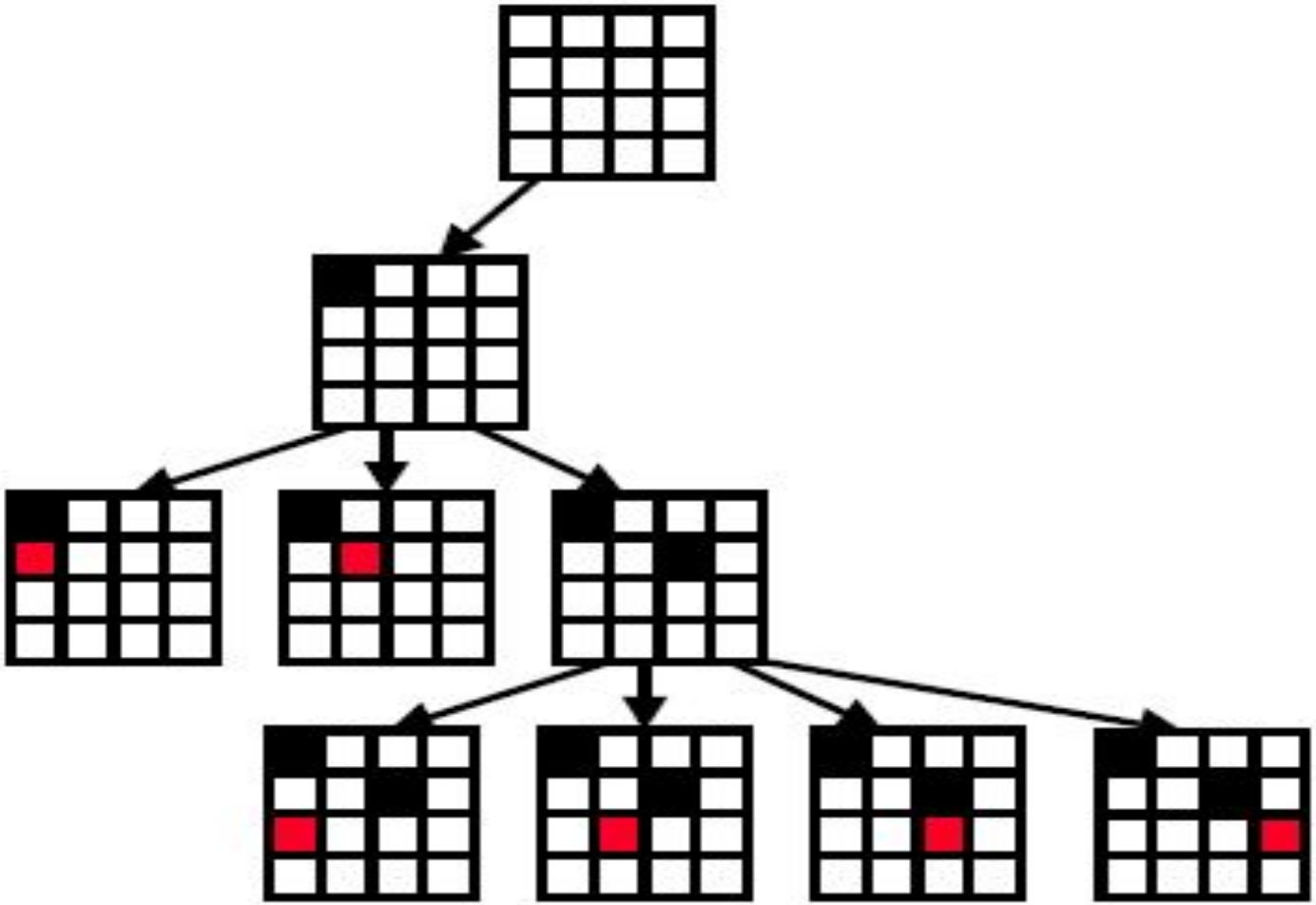
NOTE

Solution Quality is measured by the Path cost function, and an Optimal solution has the lowest path cost among all solutions

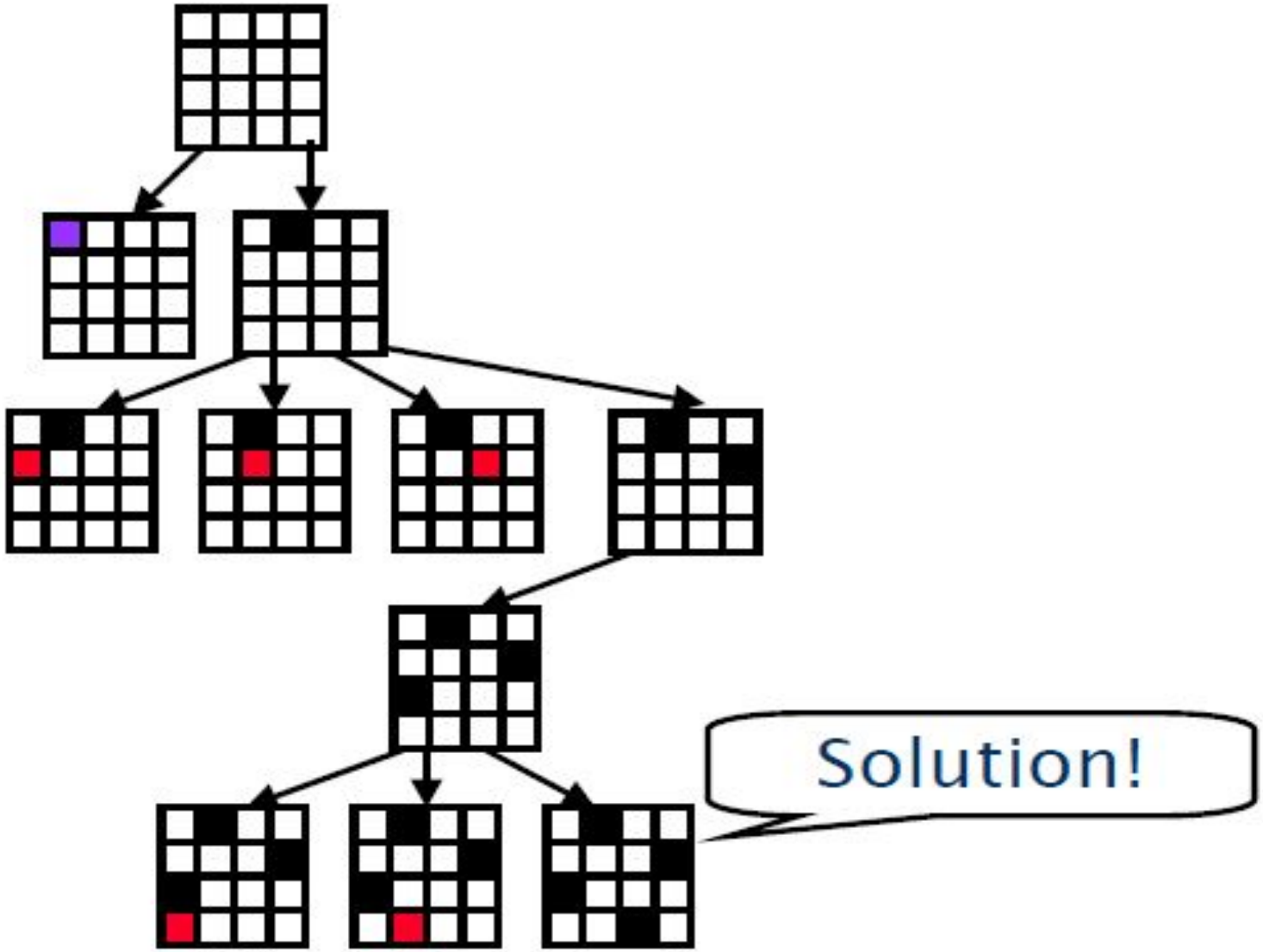
4 Queens Problem Example



4 Queens Problem Example……



4 Queens Problem Example……



The 8- Puzzle Example

State: The location of the eight tiles, and the blank

2	8	3
1	6	4
7	-	5

Initial State

1	2	3
8	-	4
7	6	5

Goal State

Initial State: {(2,0), (8,1), (3,2), (1,3), (6,4), (4,5), (7,6), (-,7), (5,8)}

Successor Function: four actions (blank moves Left, Right, Up, Down).

Goal Test: determine a given state is a goal state.

Path Cost: each step costs 1

Solution: {(1,0),(2,1),(3,2) ,(8,3) ,(-,4) ,(4,5) ,(7,6) ,(6,7) ,(5,8)}

2	8	3
1	6	4
7	-	5

2	8	3
1	-	4
7	6	5

2	-	3
1	8	4
7	6	5

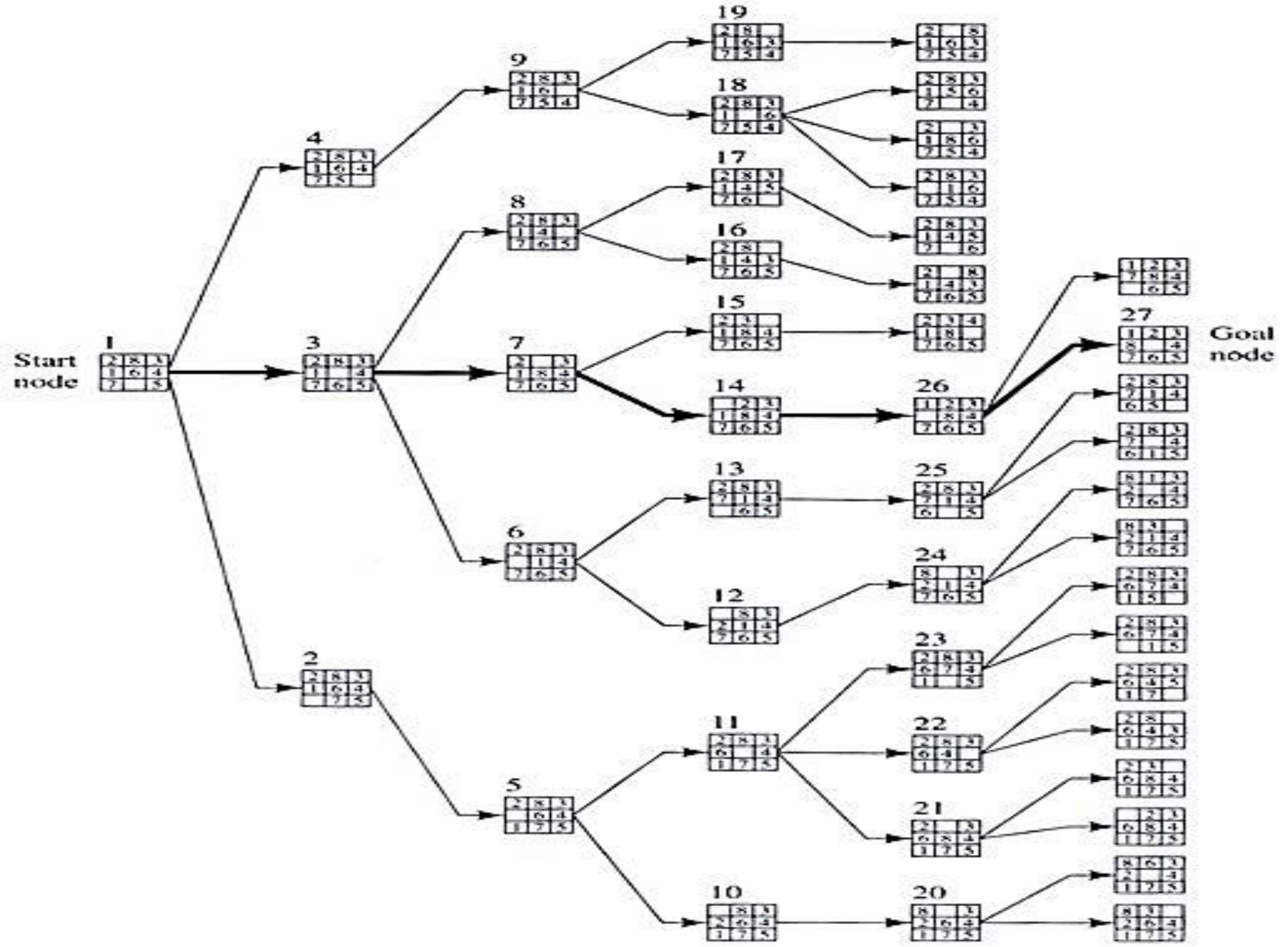
-	2	3
1	8	4
7	6	5

1	2	3
-	8	4
7	6	5

1	2	3
8	-	4
7	6	5

1	2	3
8	-	4
7	6	5

Goal

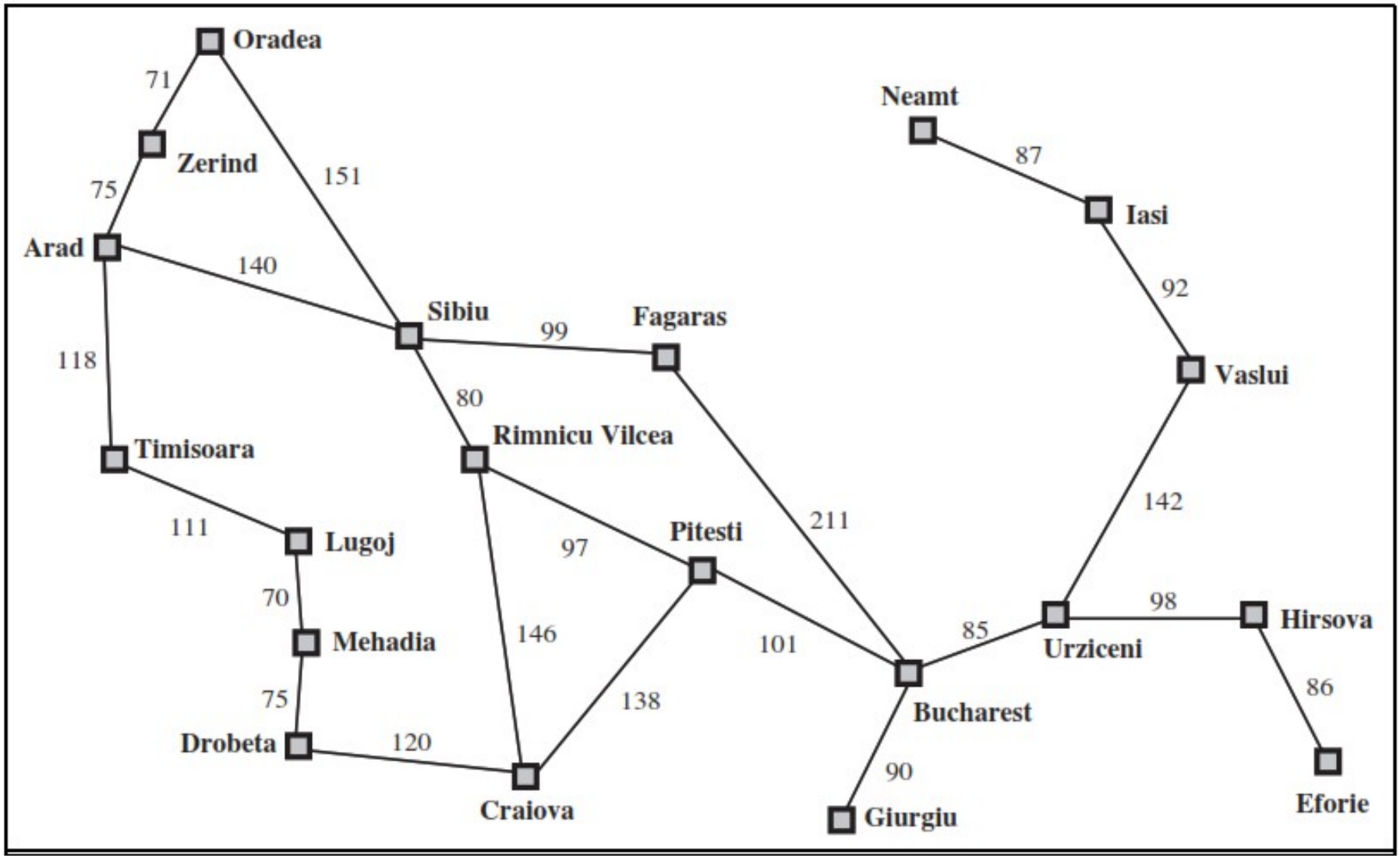

Start node
Goal node
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27

A Touring Agent Problem

(SCENARIO)

Imagine an agent in the city of Arad, Romania, enjoying a touring holiday. now, suppose the agent has a non-refundable ticket to fly out of Bucharest the following day. In that case, it makes sense for the agent to adopt the **Goal** of getting to Bucharest

Problem Formulation (The Romania Example)



Solution: a sequence of actions leading from the initial state to a goal state

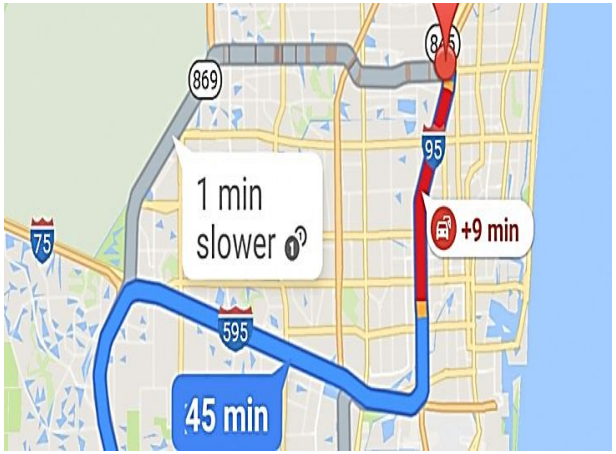
{Arad Sibiu Rimnicu Vilcea Pitesti Bucharest}

Problems

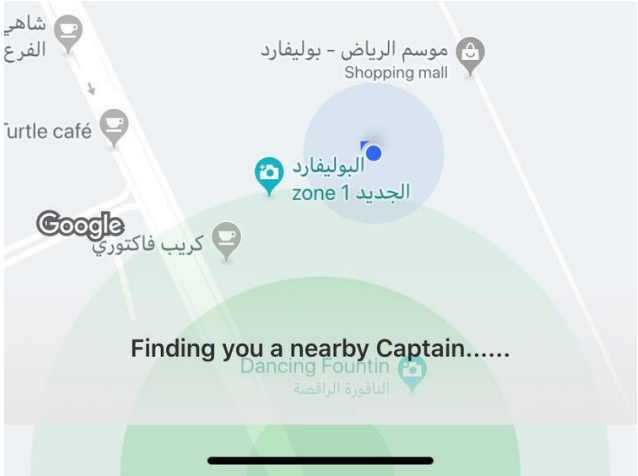
- Vacuum Cleaner
- 8-Puzzle
- 8-Queens Problem
- Robotic Assembly
- VLSI Layout

Problem Formulation (Real-life Applications)

Car Navigation



Route Finding Problem



Nearby Captain

- States
  - locations
- Initial state
  - starting point
- Successor function (operators)
  - move from one location to another
- Goal test
  - arrive at a certain location
- Path cost
  - may be quite complex
    - money, time, travel comfort, scenery,

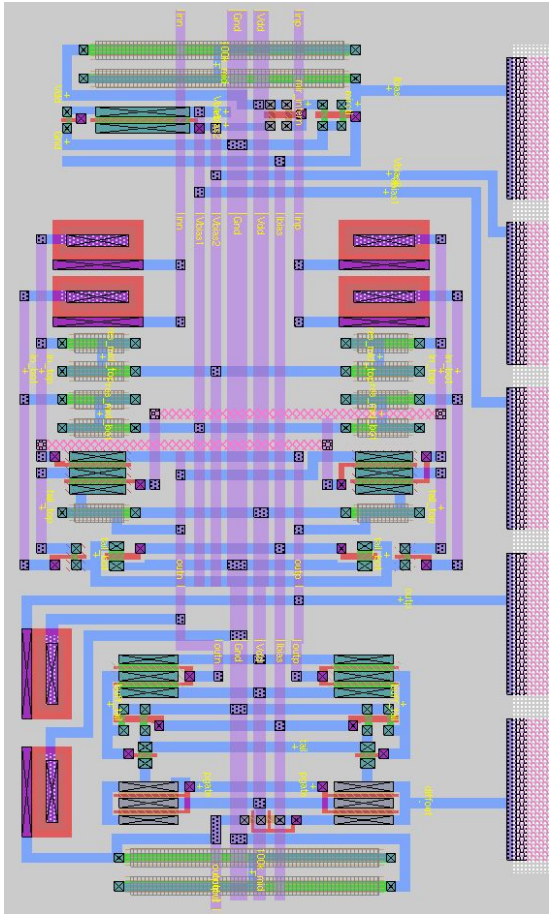
Problem Formulation (Real-life Applications)

Travel Salesperson Problem

- States
  - locations / cities
  - illegal states
    - each city may be visited only once
    - visited cities must be kept as state information
- Initial state
  - starting point
  - no cities visited
- Successor function (operators)
  - move from one location to another one
- Goal test
  - all locations visited
  - agent at the initial location
- Path cost
  - distance between locations

Problem Formulation (Real-life Applications)

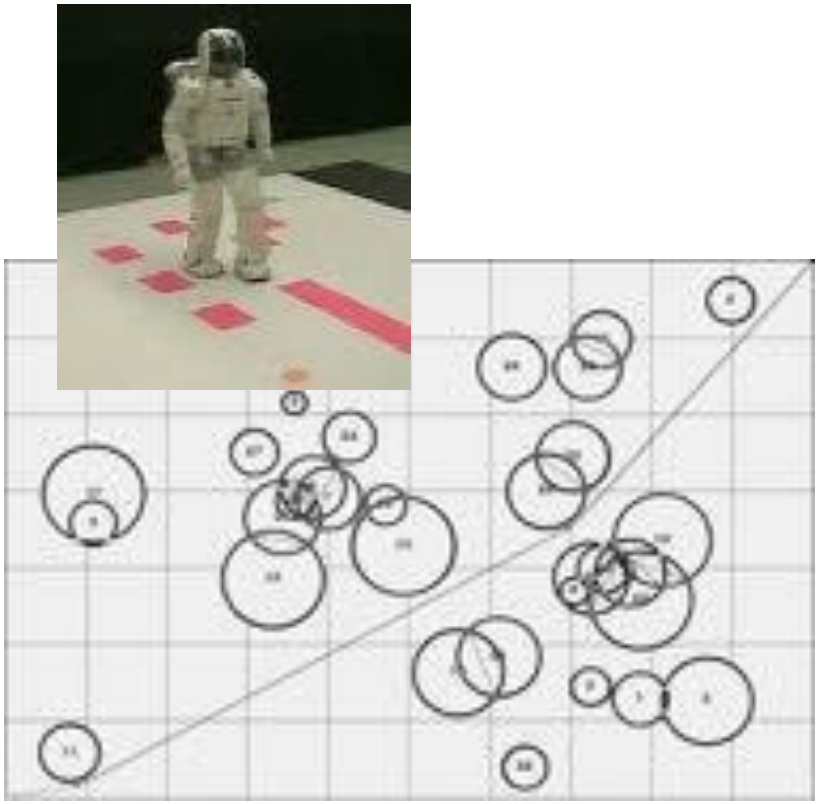
VLSI layout Problem



- States
  - positions of components, wires on a chip
- Initial state
  - incremental: no components placed
  - complete-state: all components placed (e.g. randomly, manually)
- Successor function (operators)
  - incremental: place components, route wire
  - complete-state: move component, move wire
- Goal test
  - all components placed
  - components connected as specified
- Path cost
  - maybe complex
    - component distance, capacity, number of connections per

Problem Formulation (Real-life Applications)

Robot Navigation



- States
  - locations
  - position of actuators
- Initial state
  - start position (dependent on the task)
- Successor function (operators)
  - movement, actions of actuators
- Goal test
  - task-dependent
- Path cost
  - maybe very complex
    - distance, energy consumption

Problem Formulation (Real-life Applications)

Automatic Assembly Sequencing

- States
  - location of components
- Initial state
  - no components assembled
- Successor function (operators)
  - place component
- Goal test
  - system fully assembled
- Path cost
  - number of moves

Search Algorithms

Uninformed Search

Informed Search

Uninformed Search

- breadth-first
- uniform-cost search
- depth-first
- depth-limited search
- iterative deepening
- bi-directional search

(blind search)

- number of steps, path cost unknown
- agent knows when it reaches a goal

Informed Search

- Greedy best-first
- A*

(heuristic search)

- agent has background information about the problem

Uninformed (Blind Search)

- The Uninformed Search does not contain any domain knowledge such as closeness or location of goal.

- It operates in a brute force way, as it only includes information about how to traverse the tree and how to identify leaf and goal nodes.

- Uninformed Search applies a way in which search tree is searched without any information, so it is called blind search.

- It examines each node until it achieves the goal node.

Evaluation of Search Strategies

A search strategy is defined by picking the order of node expansion

Strategies are evaluated along the following dimensions:

- Completeness: if there is a solution, will it be found
- Time complexity: How long does it takes to find the solution
- Space complexity: memory required for the search
- Optimality: will the best solution be found

Time and space complexity are measured in terms of

- *b:* maximum branching factor of the search tree
- *d:* depth of the least-cost solution
- *m*: maximum depth of the state space (may be ∞)

Searching for Solutions

- Traversal of the search space
  - From the initial state to a goal state.
  - Legal sequence of actions as defined by successor function.

- General procedure
  - Check for goal state
  - Expand the current state
    - Determine the set of reachable states
    - Return “failure” if the set is empty
  - Select one from the set of reachable states
  - Move to the selected state

- A search tree is generated
  - Nodes are added as more states are visited

Search Terminology

Search Tree

- Generated as the search space is traversed
  - The search space itself is not necessarily a tree, frequently it is a graph
  - The tree specifies possible paths through the search space
- Expansion of nodes
  - As states are explored, the corresponding nodes are expanded by
    applying the successor function
    - this generates a new set of (child) nodes
  - The *fringe* (frontier/queue) is the set of nodes not yet visited
    - newly generated nodes are added to the fringe
- Search strategy
  - Determines the selection of the next node to be expanded
  - Can be achieved by ordering the nodes in the fringe
    - e.g. queue (FIFO), stack (LIFO), “best” node w.r.t. some measure (cost)

Search Tree Vs Graph Tree

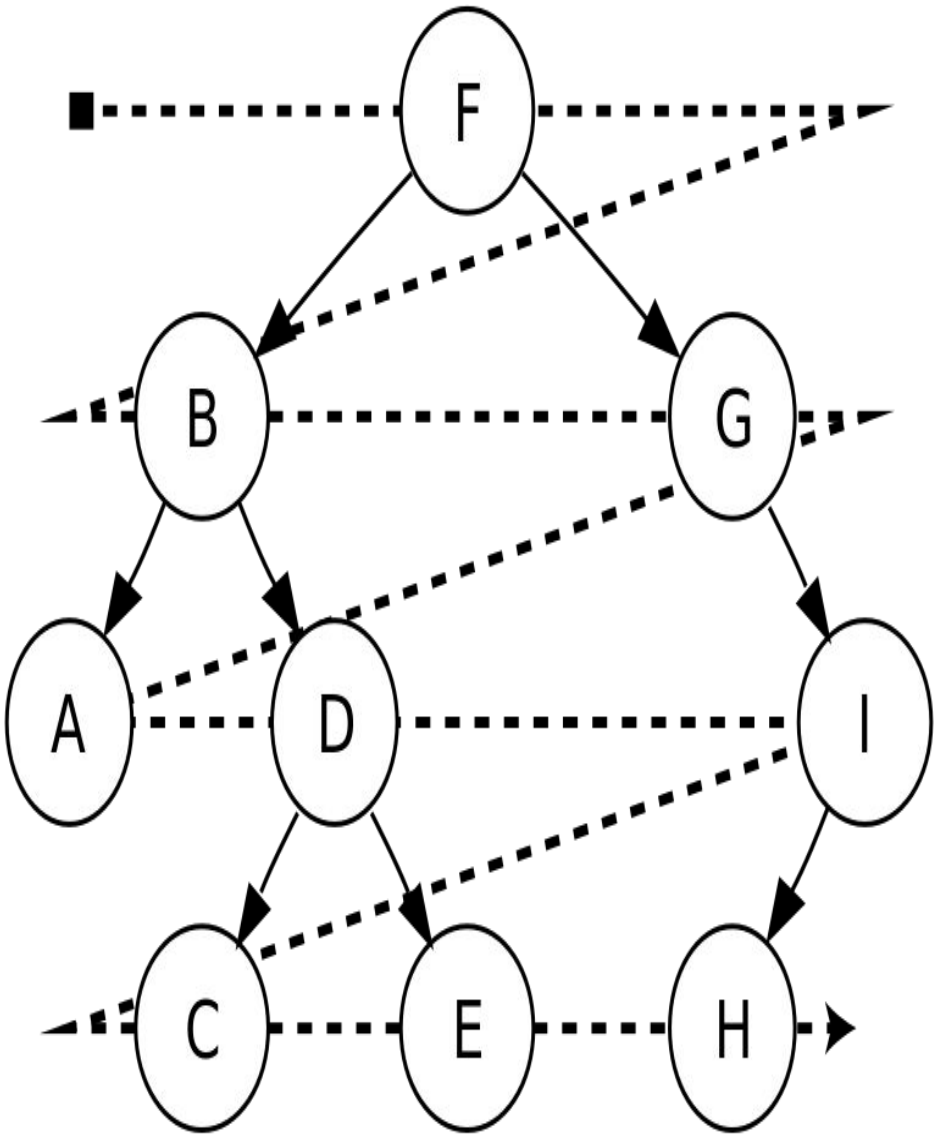
BASIS FOR COMPARISON	TREE	GRAPH
Path	Only one between two vertices.	More than one path is allowed.
Root node	It has exactly one root node.	Graph doesn't have a root node.
Loops	No loops are permitted.	Graph can have loops.
Complexity	Less complex	More complex comparatively
Traversal techniques	Pre-order, In-order and Post-order.	Breadth-first search and depth-first search.
Number of edges	n-1 (where n is the number of nodes)	Not defined
Model type	Hierarchical	Network

1. Breadth-First Search (BFS) Algorithm

Breadth-First Search

- It is the most common search strategy for traversing a tree or graph.
- This algorithm searches breadthwise in a tree or graph, so it is called breadth-first search.
- BFS algorithm starts searching from the root node of the tree and further are traversed in *level-order*, where we visit every node on a level before going to a lower level.
- It starts at the tree root (or some arbitrary node of a graph)
- BFS is implemented using FIFO Queue data structure.

Example: BFS

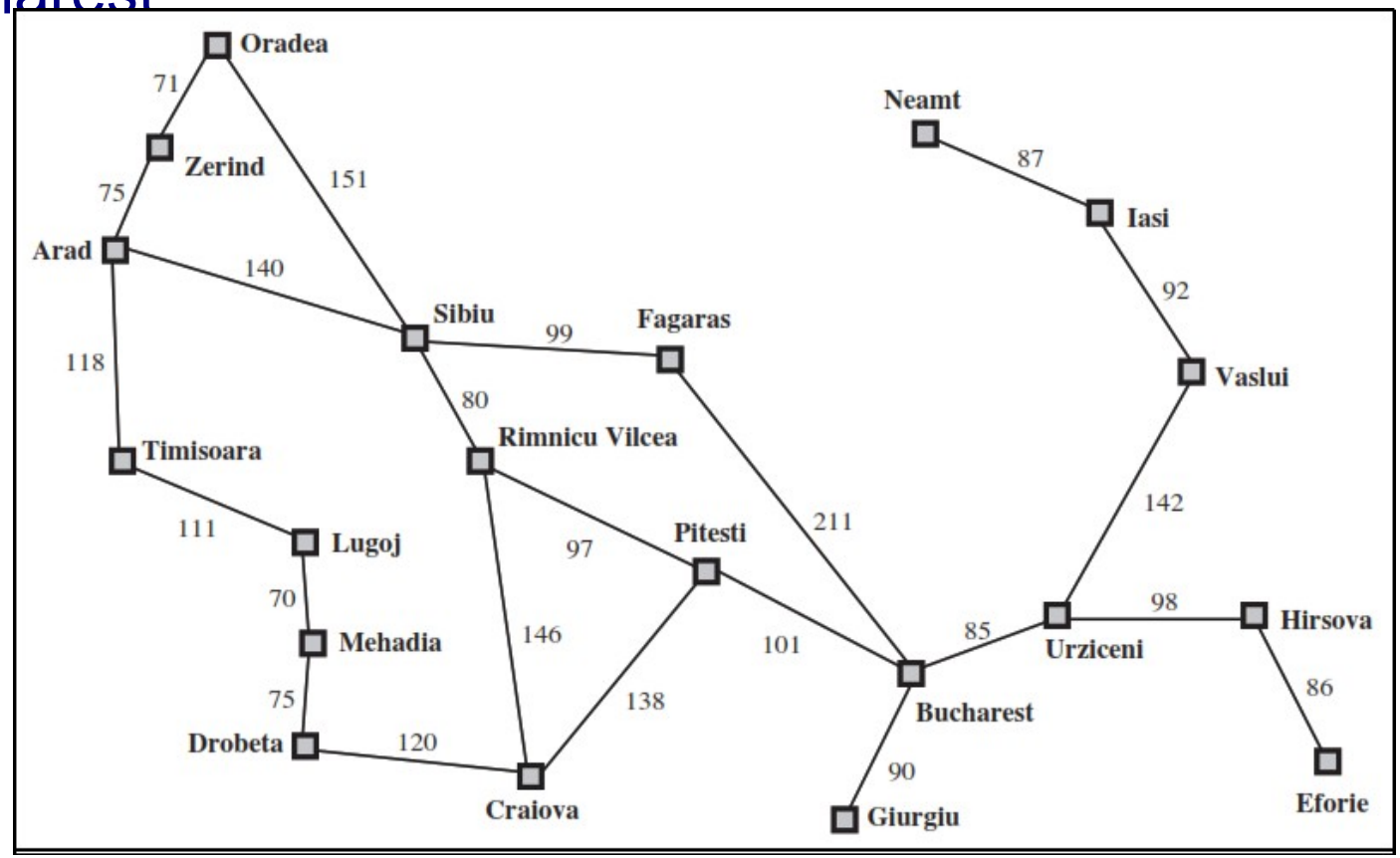


For example, after searching F, then B, then G, the search proceeds with A,D, I, C, E,H

Node are explored in the Level order F,B,G,A,D,I,C,E,H

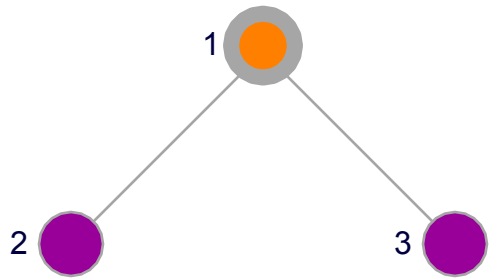
Example: Romania

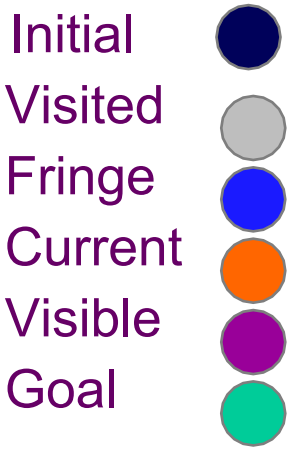
You are in Arad and want to go to Bucharest



 How to design an intelligent agent to find the way between 2 cities?

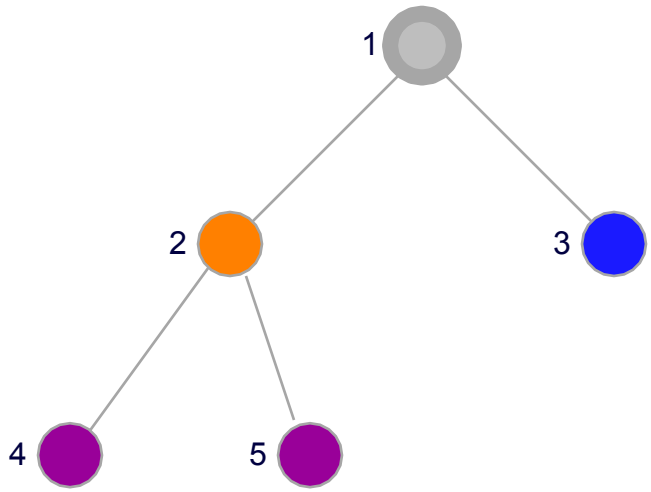
Breadth-First Snapshot 1

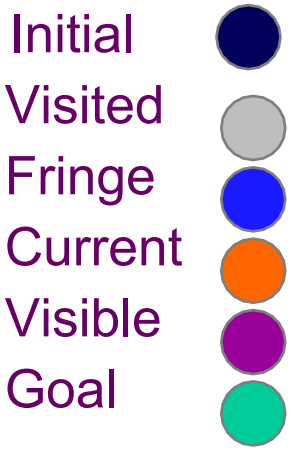




Fringe: [] + [2,3]

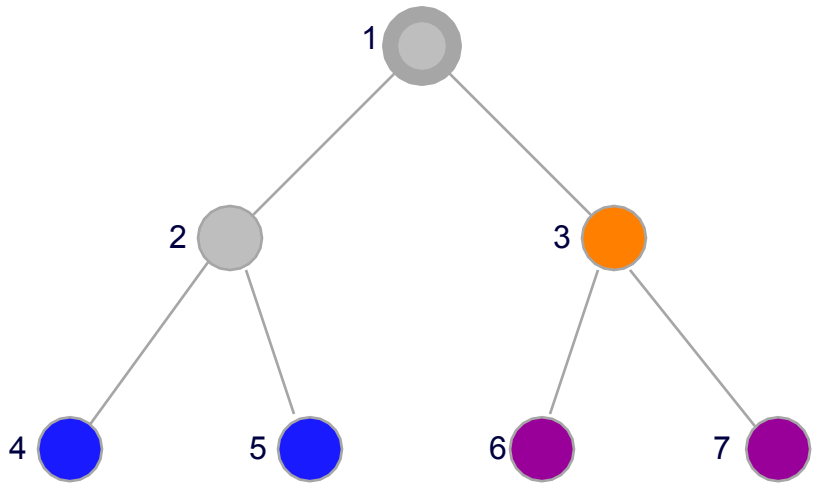
Breadth-First Snapshot 2

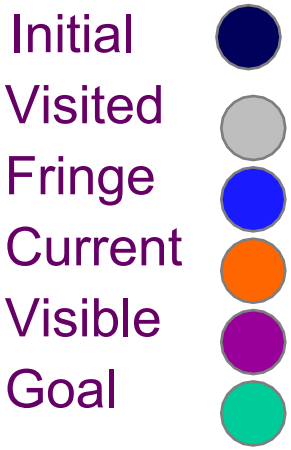




Fringe: [3] + [4,5]

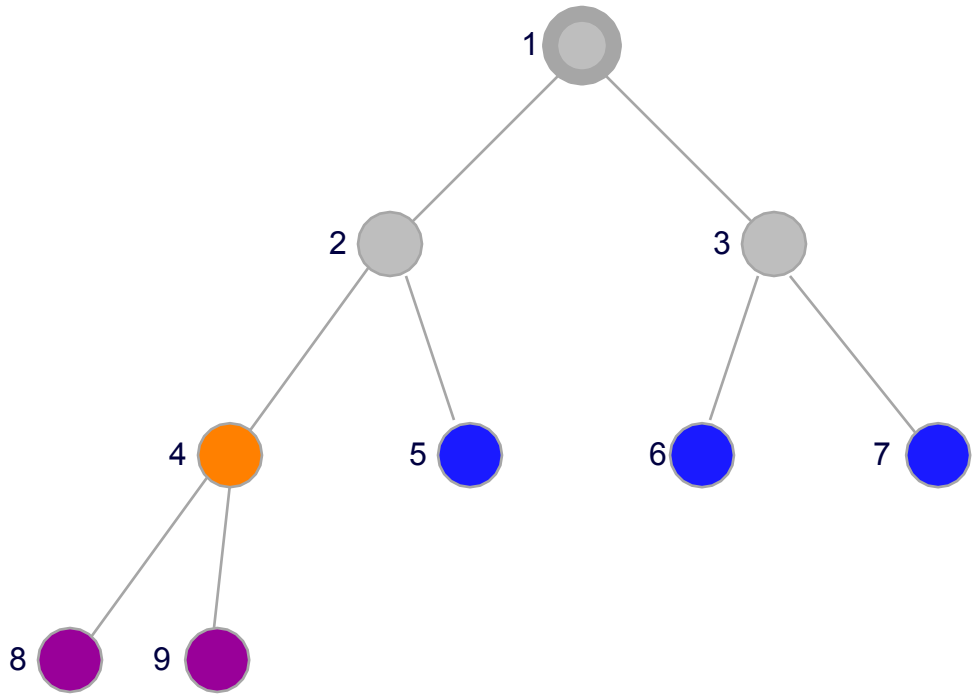
Breadth-First Snapshot 3

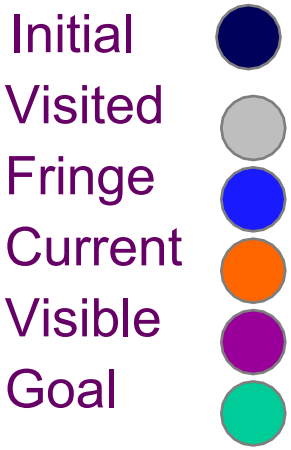




Fringe: [4,5] + [6,7]

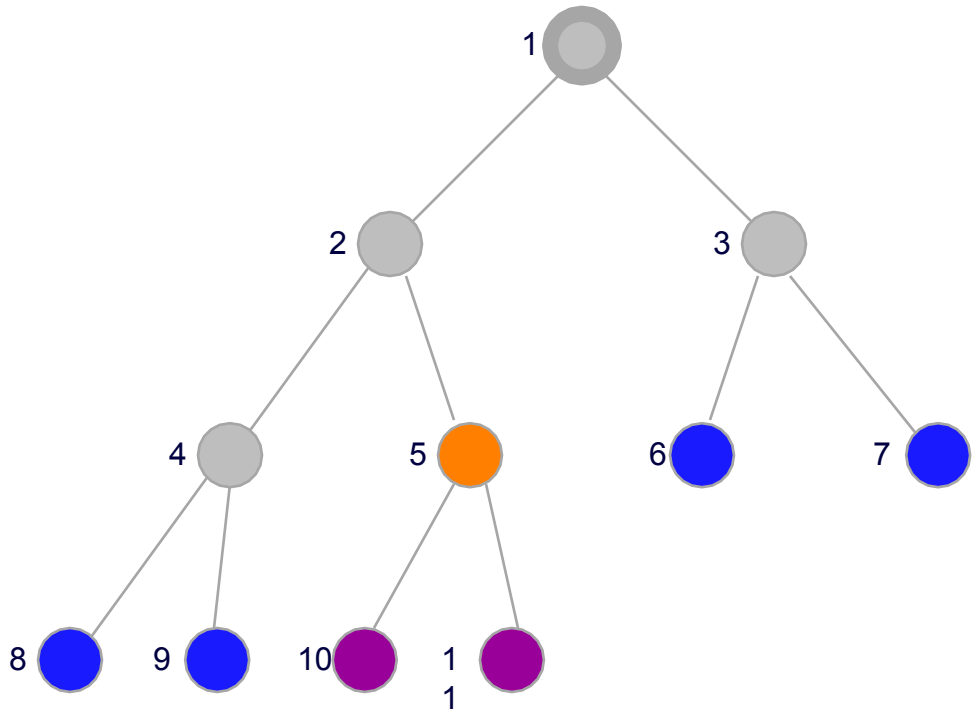
Breadth-First Snapshot 4

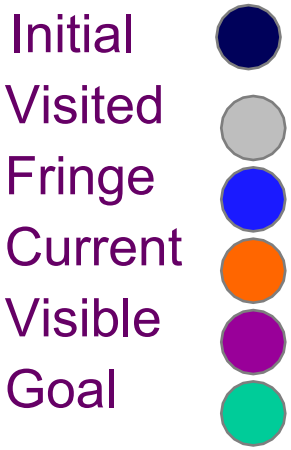




Fringe: [5,6,7] + [8,9]

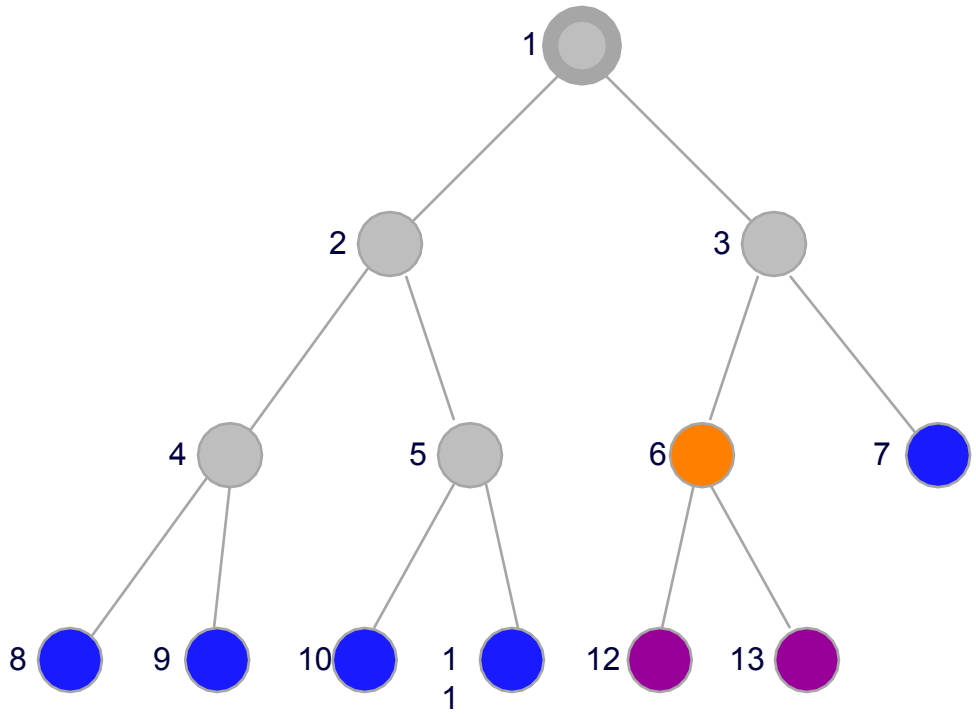
Breadth-First Snapshot 5

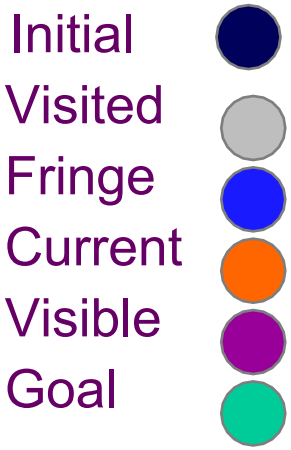




Fringe: [6,7,8,9] + [10,11]

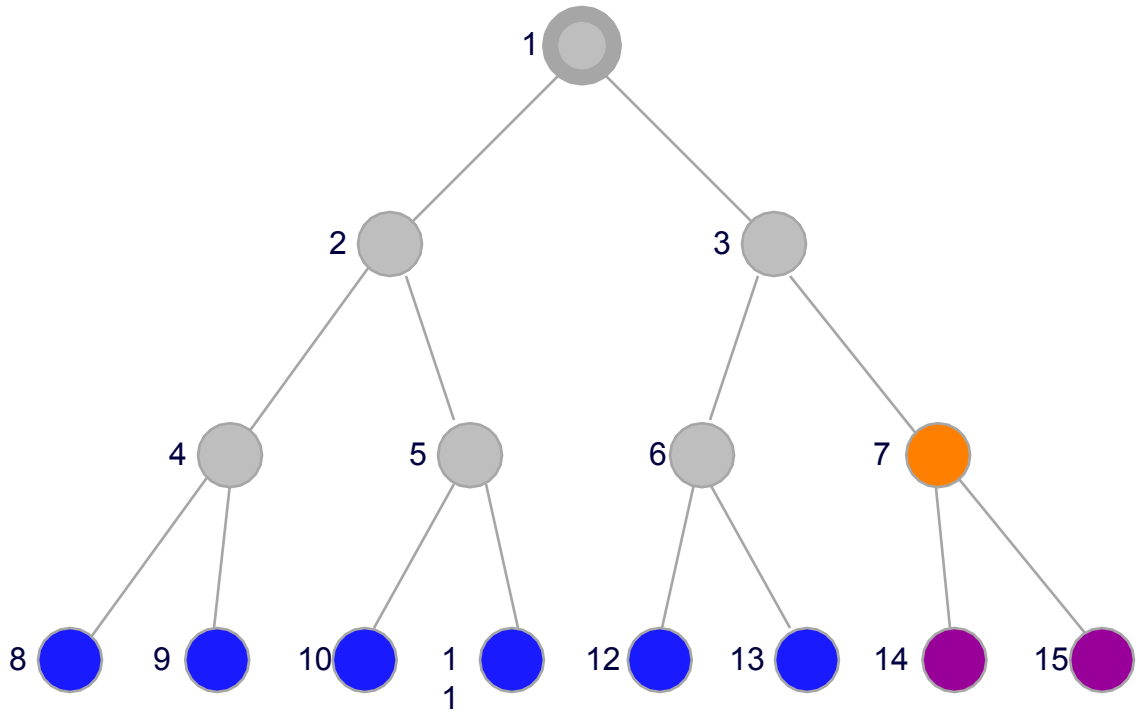
Breadth-First Snapshot 6

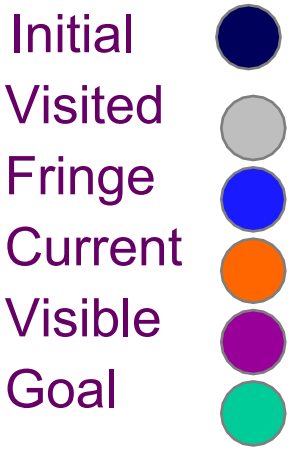




Fringe: [7,8,9,10,11] + [12,13]

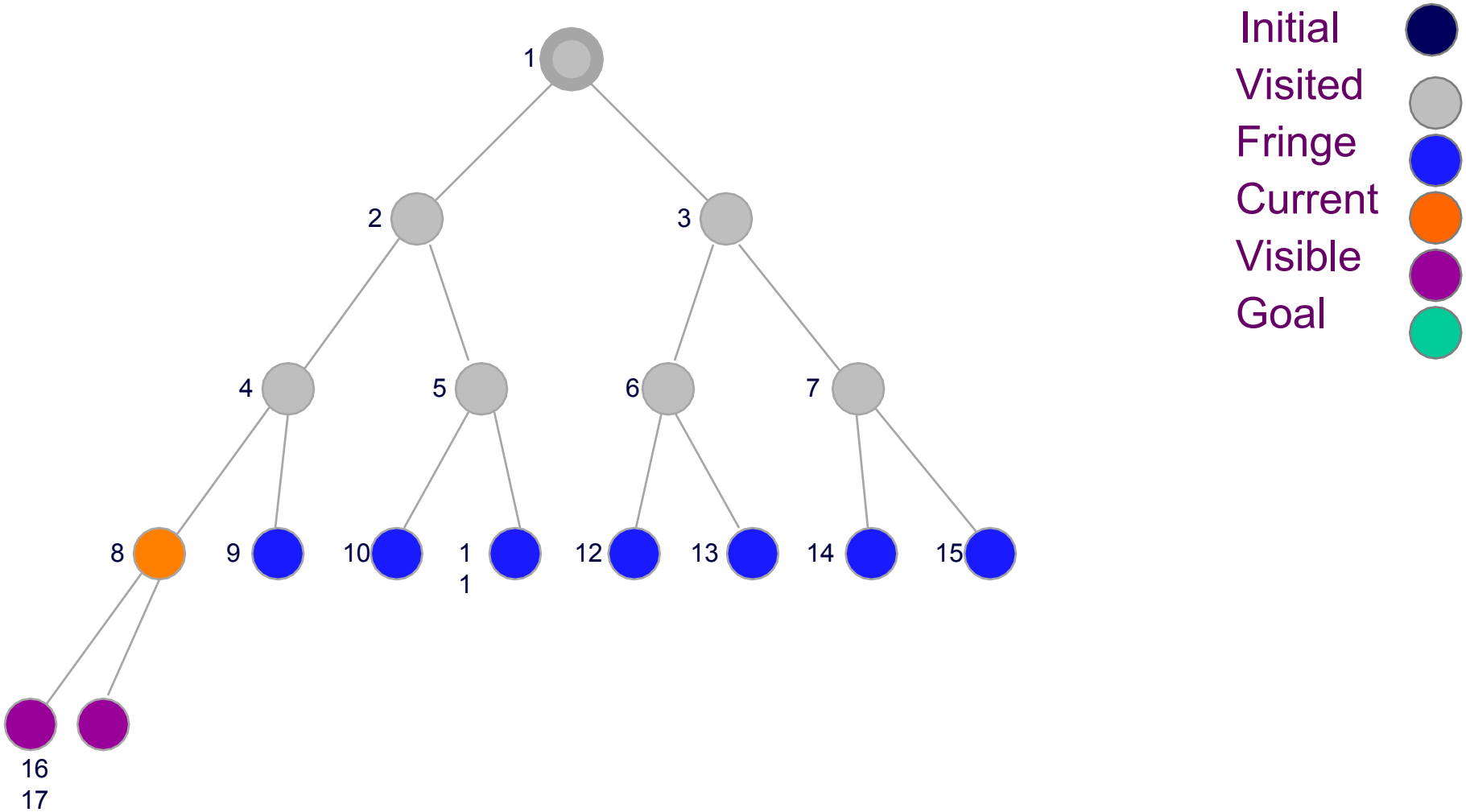
Breadth-First Snapshot 7





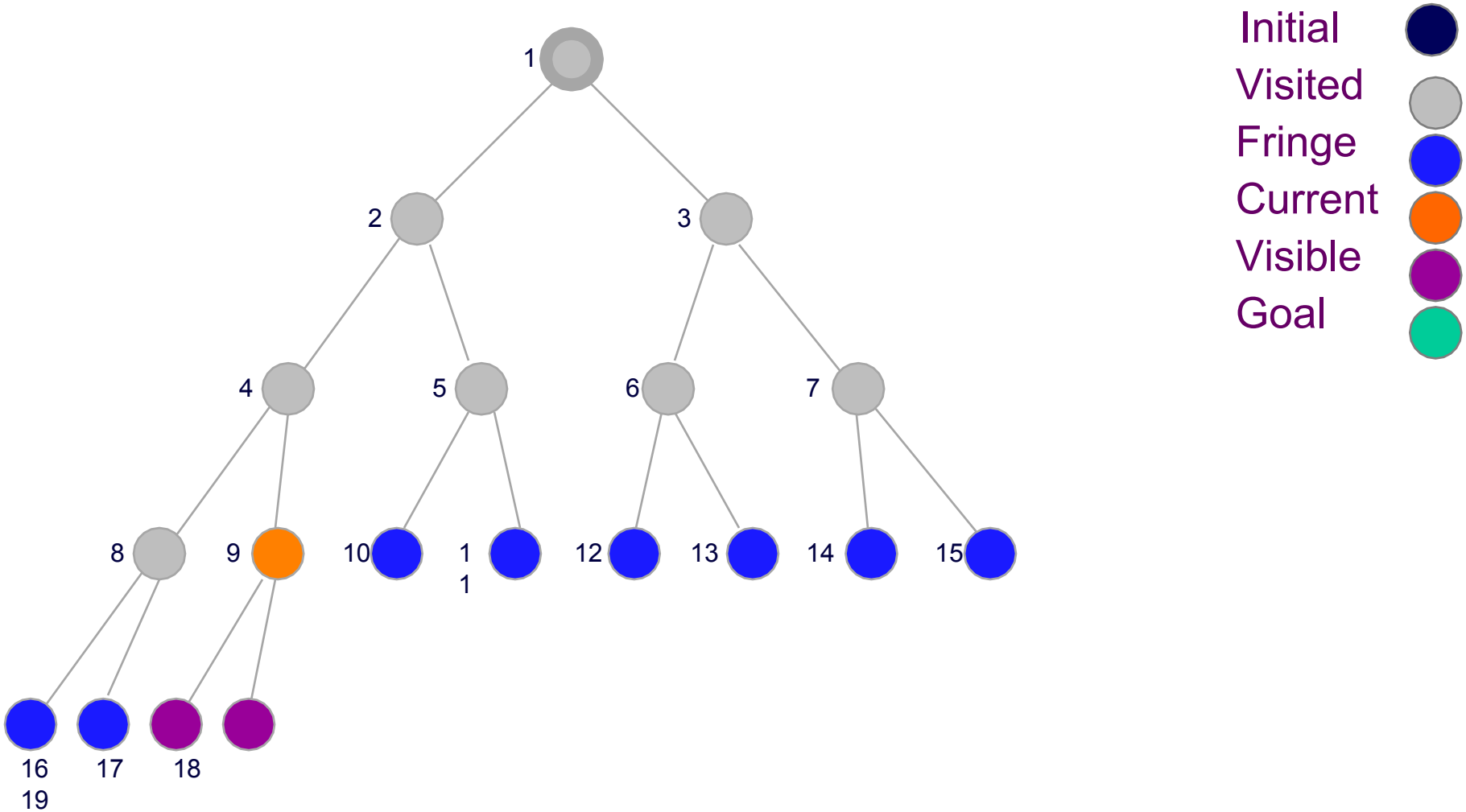
Fringe: [8,9,10,11,12,13] + [14,15]

Breadth-First Snapshot 8



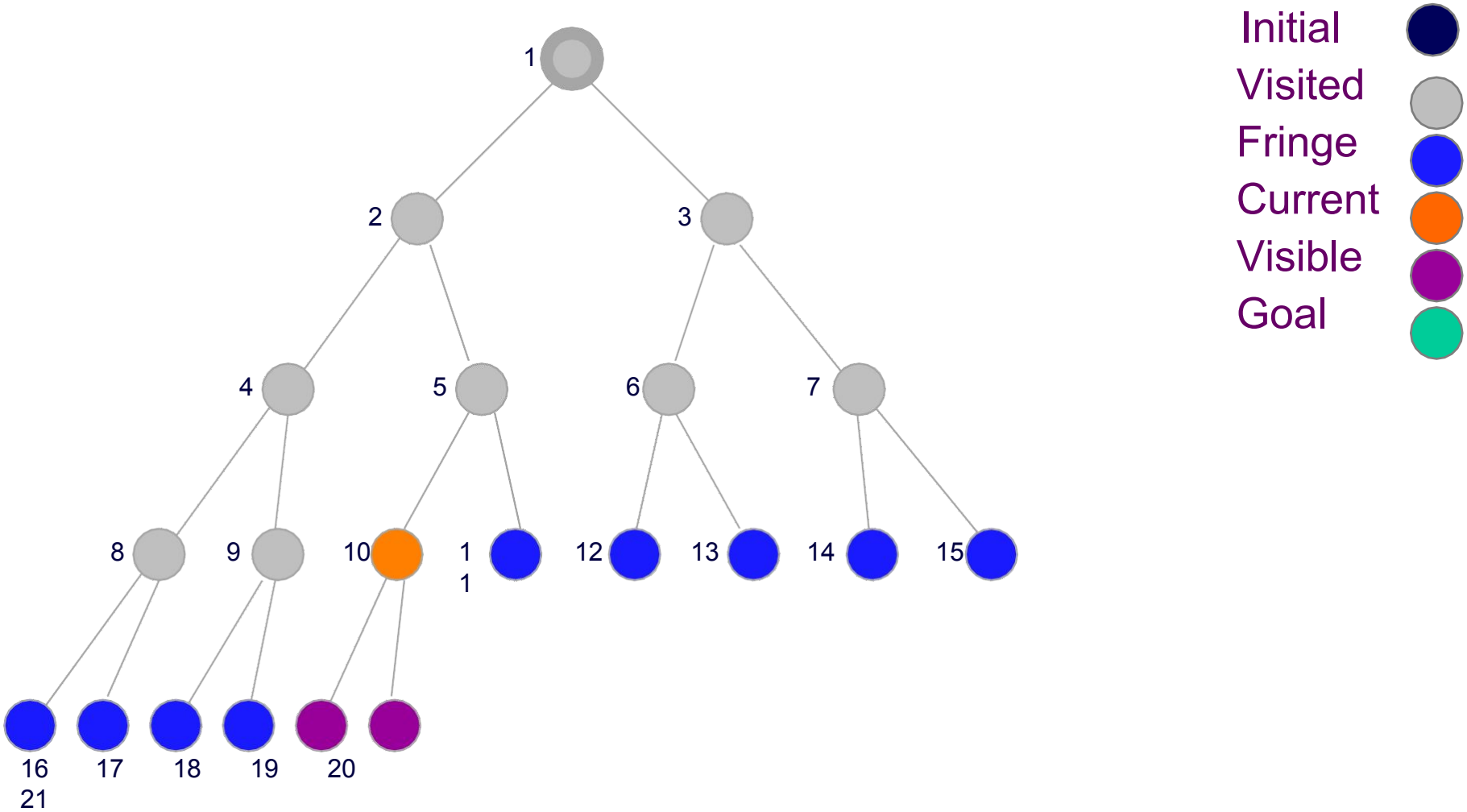
Fringe: [9,10,11,12,13,14,15] + [16,17]

Breadth-First Snapshot 9



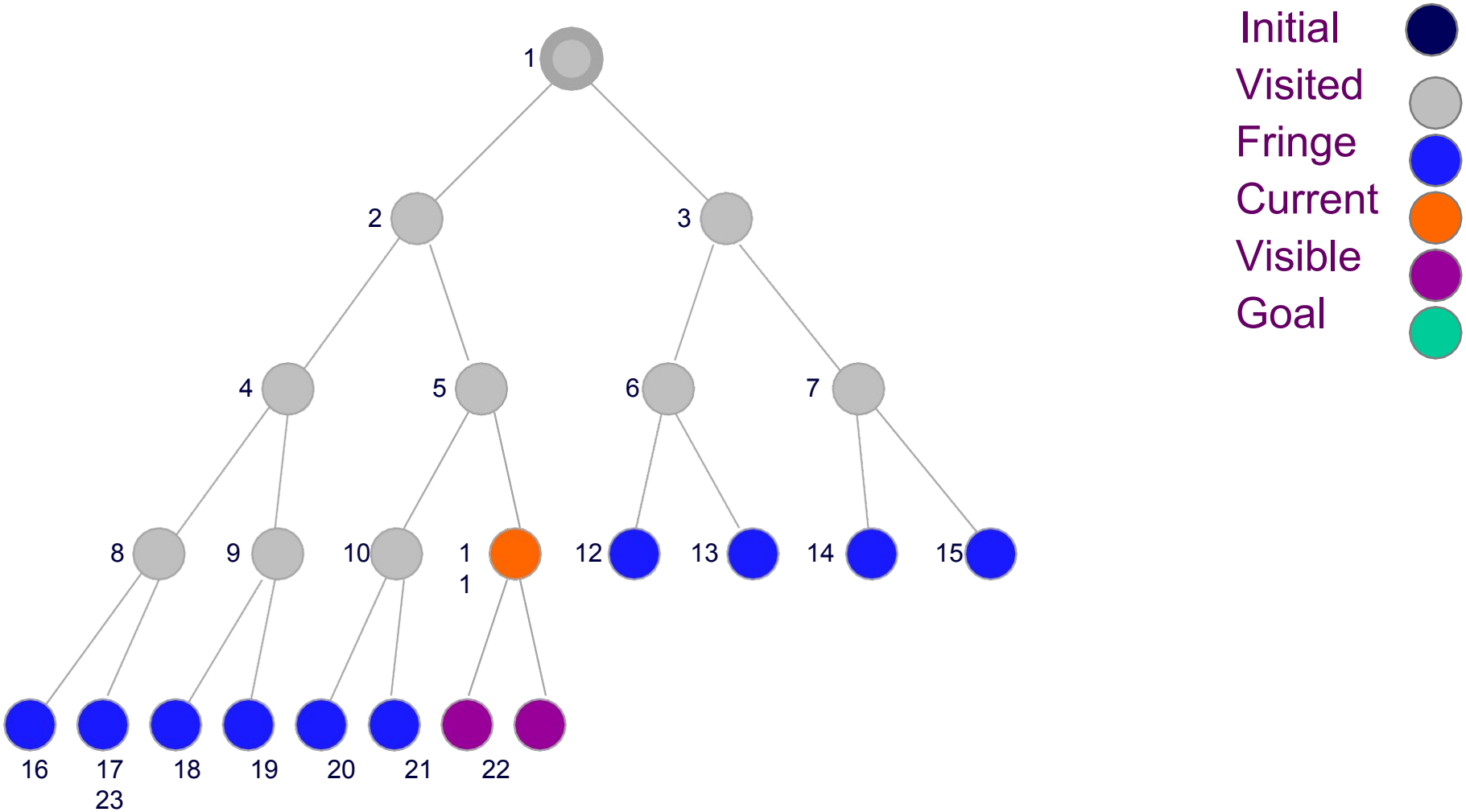
Fringe: [10,11,12,13,14,15,16,17] + [18,19]

Breadth-First Snapshot 10



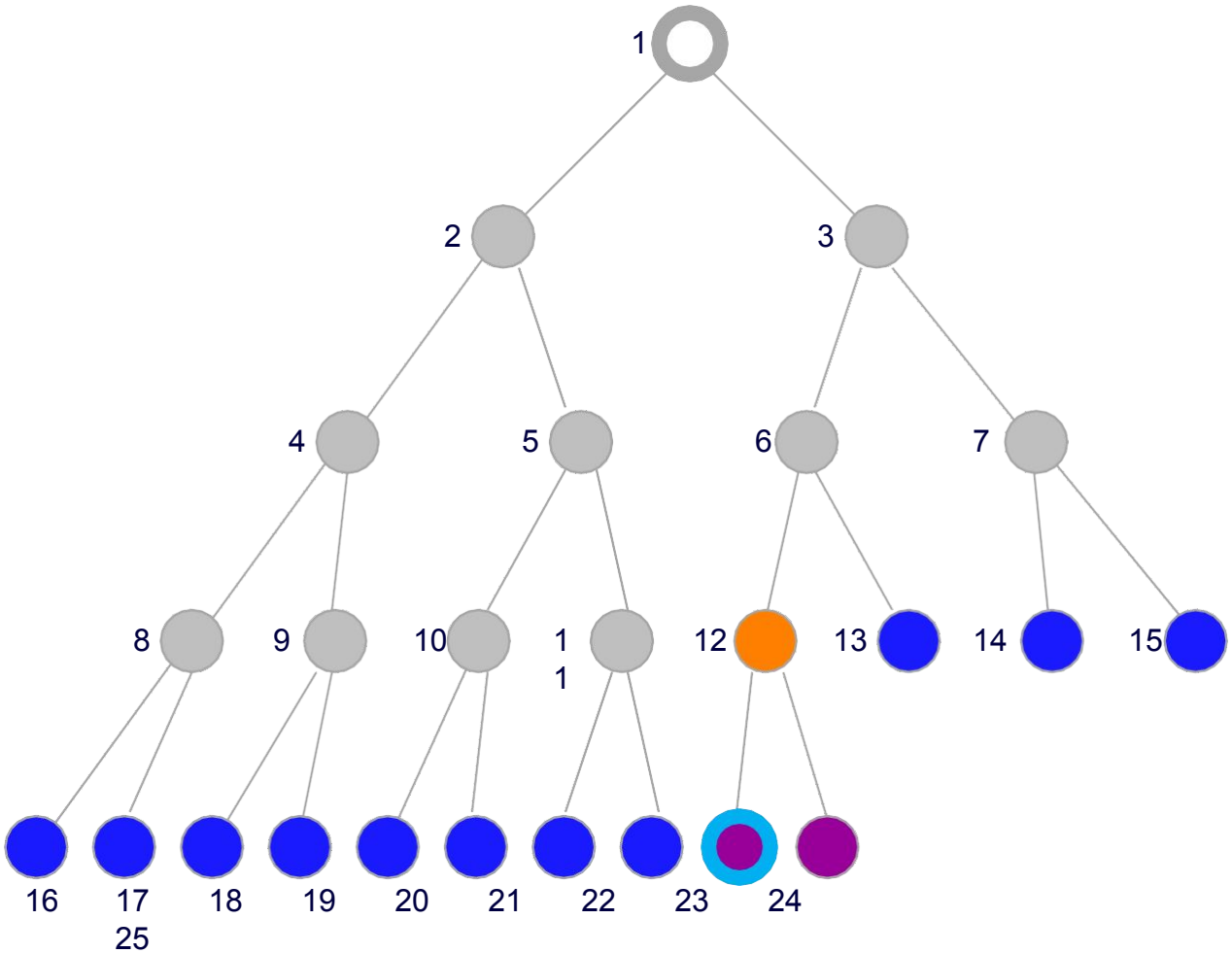
Fringe: [11,12,13,14,15,16,17,18,19] + [20,21]

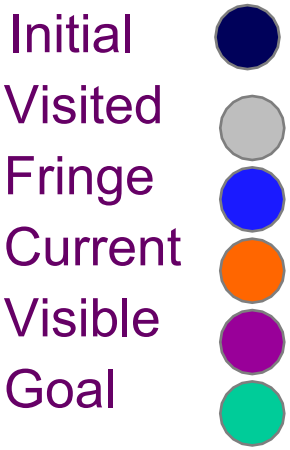
Breadth-First Snapshot 11



Fringe: [12, 13, 14, 15, 16, 17, 18, 19, 20, 21] + [22,23]

Breadth-First Snapshot 12

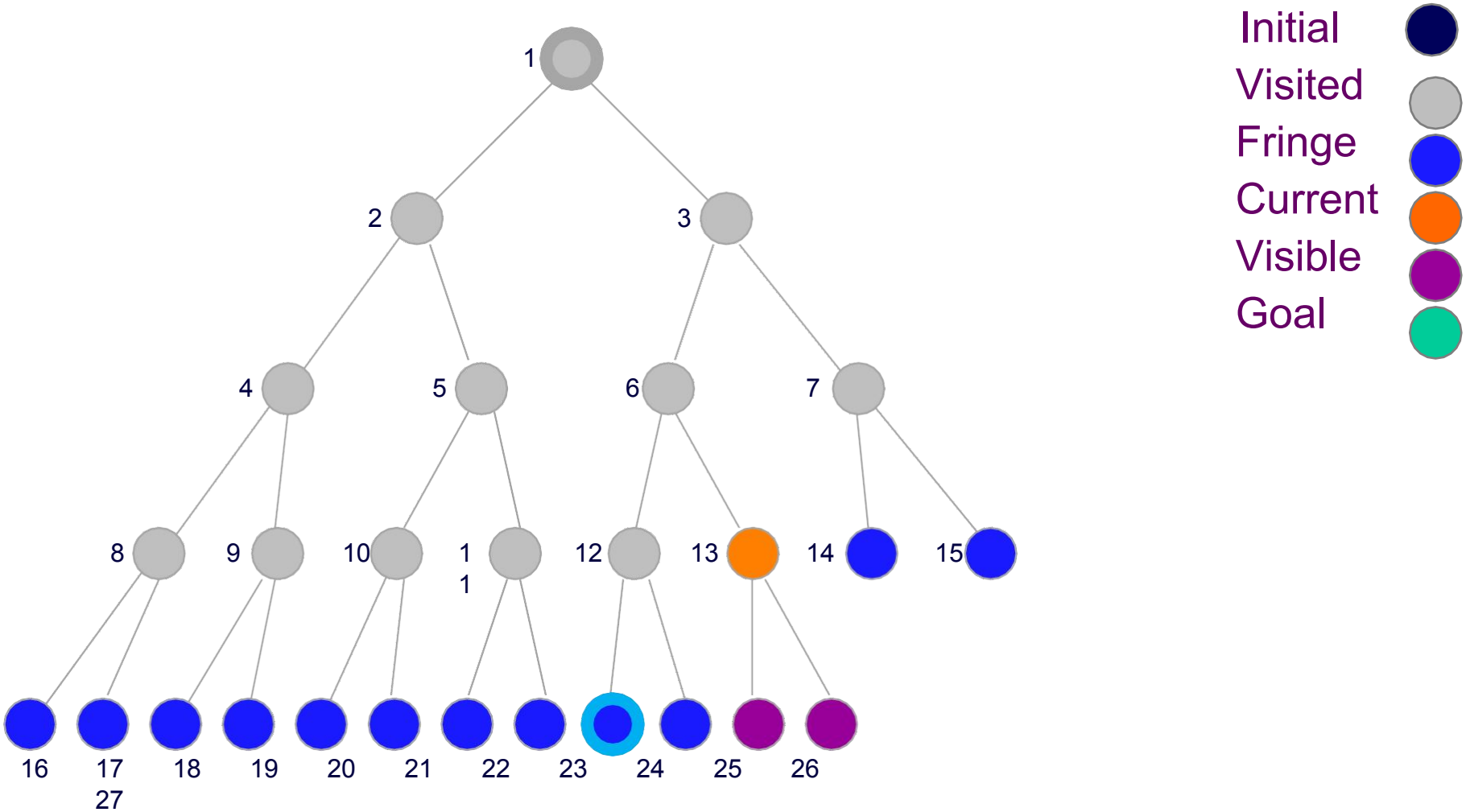




Note:
The goal node is "visible" here, but we can not perform the goal test yet.

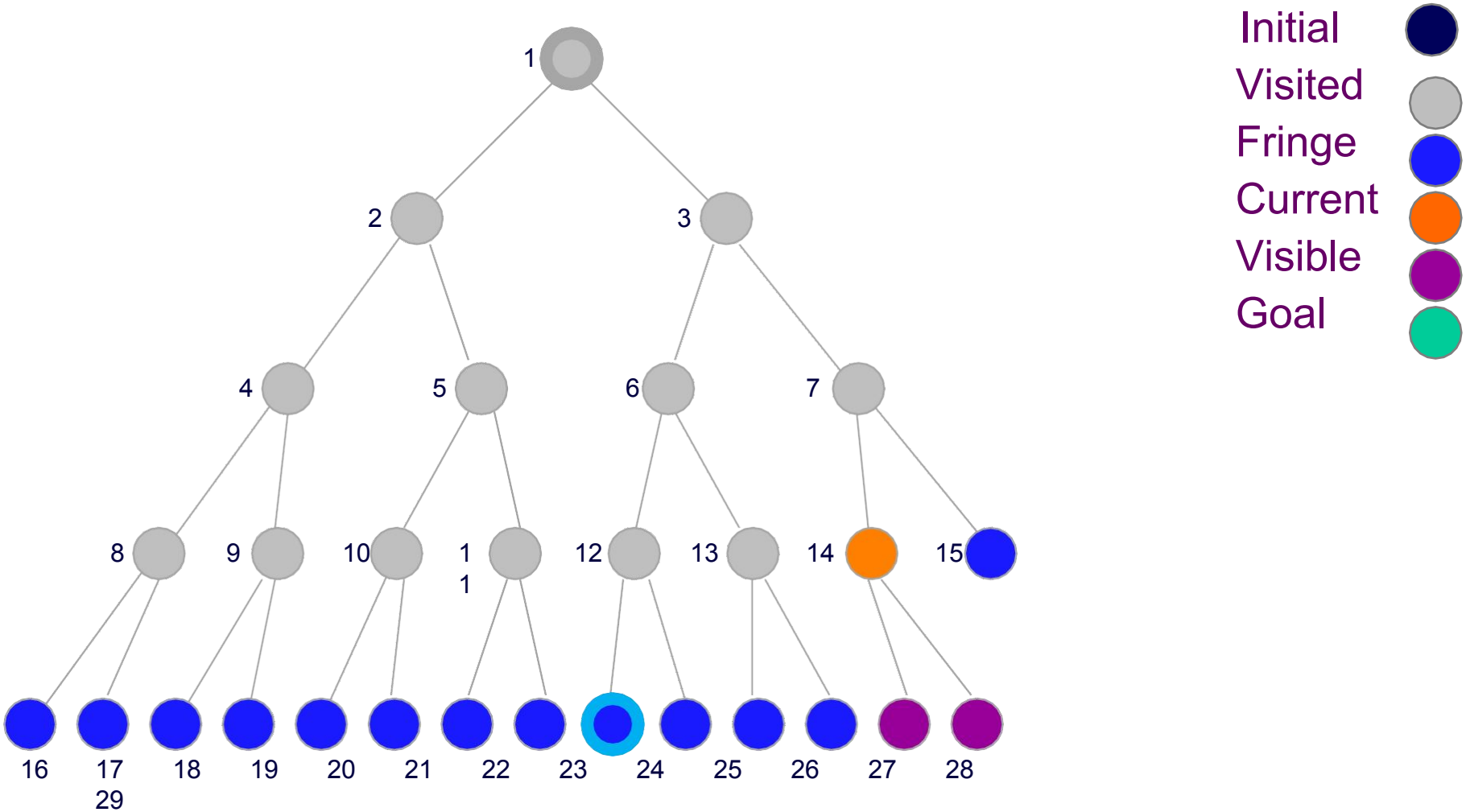
Fringe: [13,14,15,16,17,18,19,20,21] + [22,23]

Breadth-First Snapshot 13



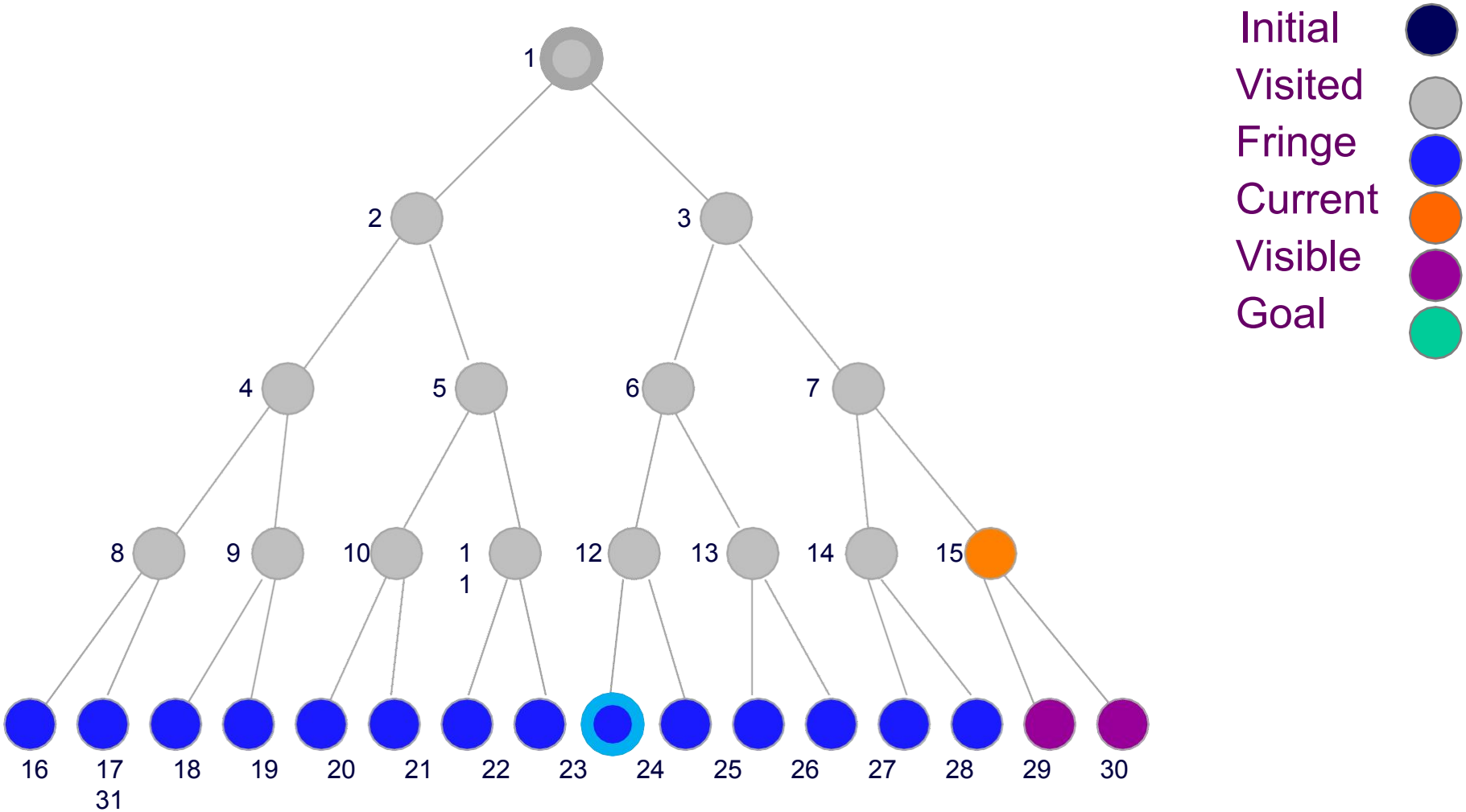
Fringe: [14,15,16,17,18,19,20,21,22,23,24,25] + [26,27]

Breadth-First Snapshot 14



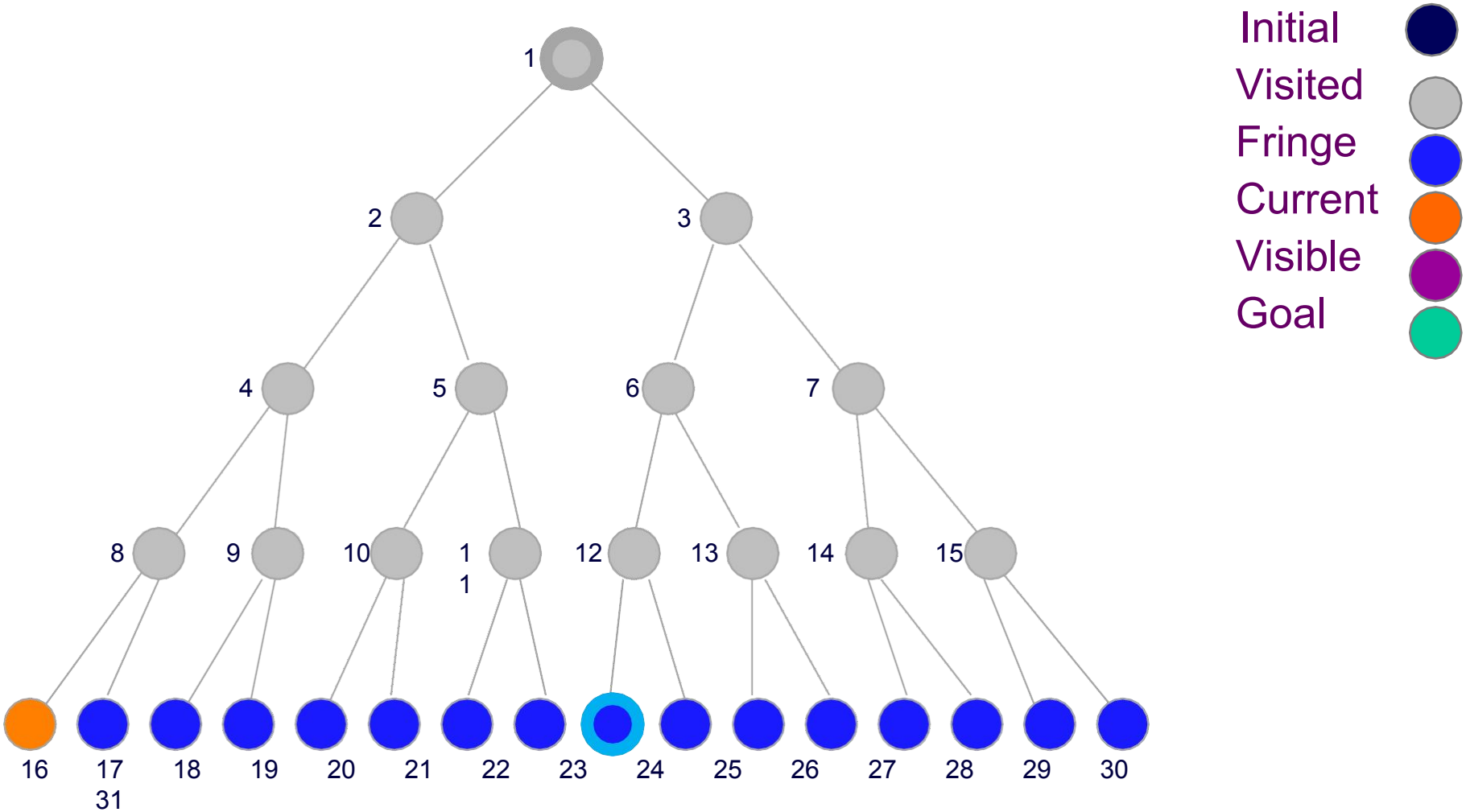
Fringe: [15,16,17,18,19,20,21,22,23,24,25,26,27] + [28,29]

Breadth-First Snapshot 15



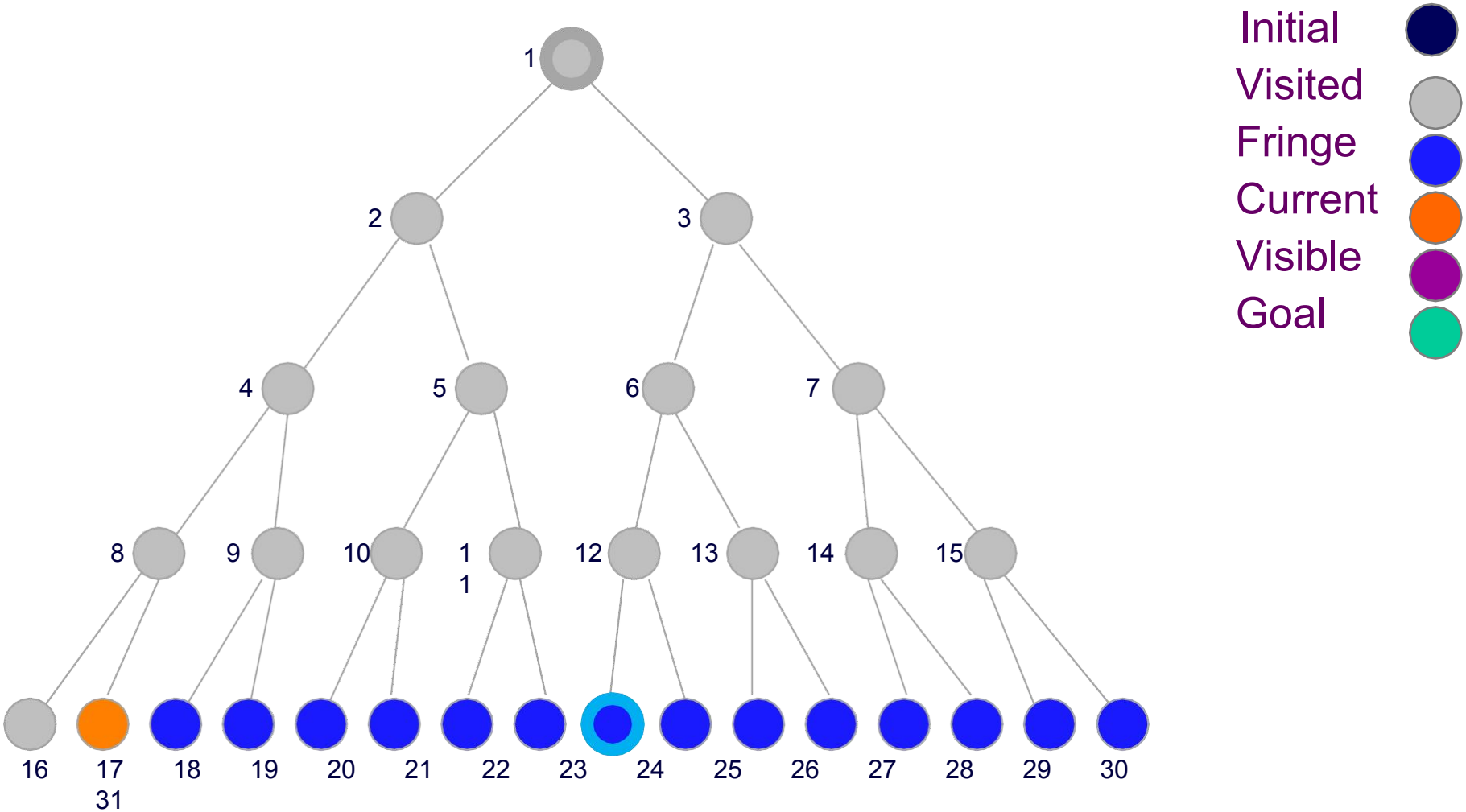
Fringe: [15,16,17,18,19,20,21,22,23,24,25,26,27,28,29] + [30,31]

Breadth-First Snapshot 16



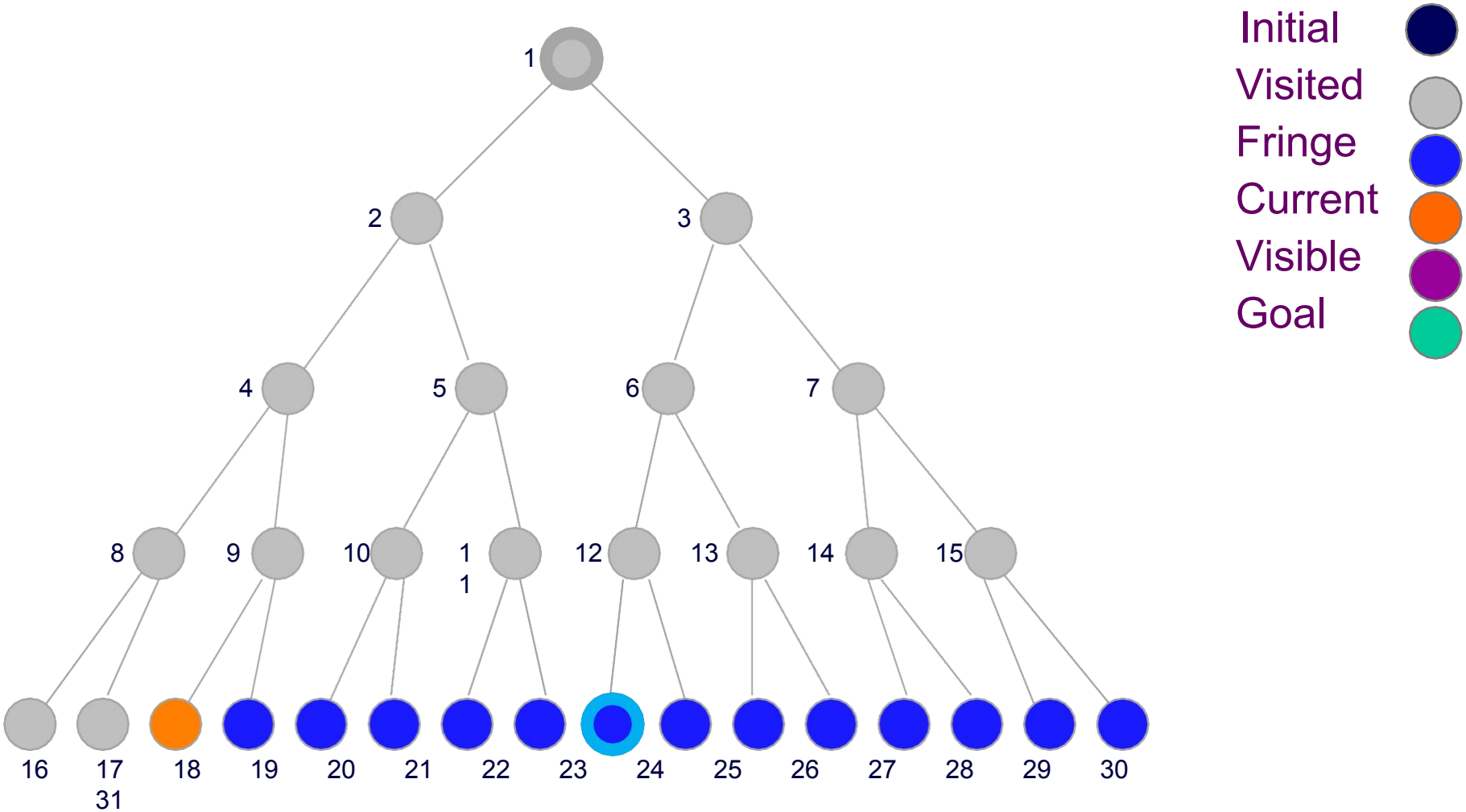
Fringe:
[17,18,19,20,21,22,23,24,25,26,27,28,29,30,31]

Breadth-First Snapshot 17



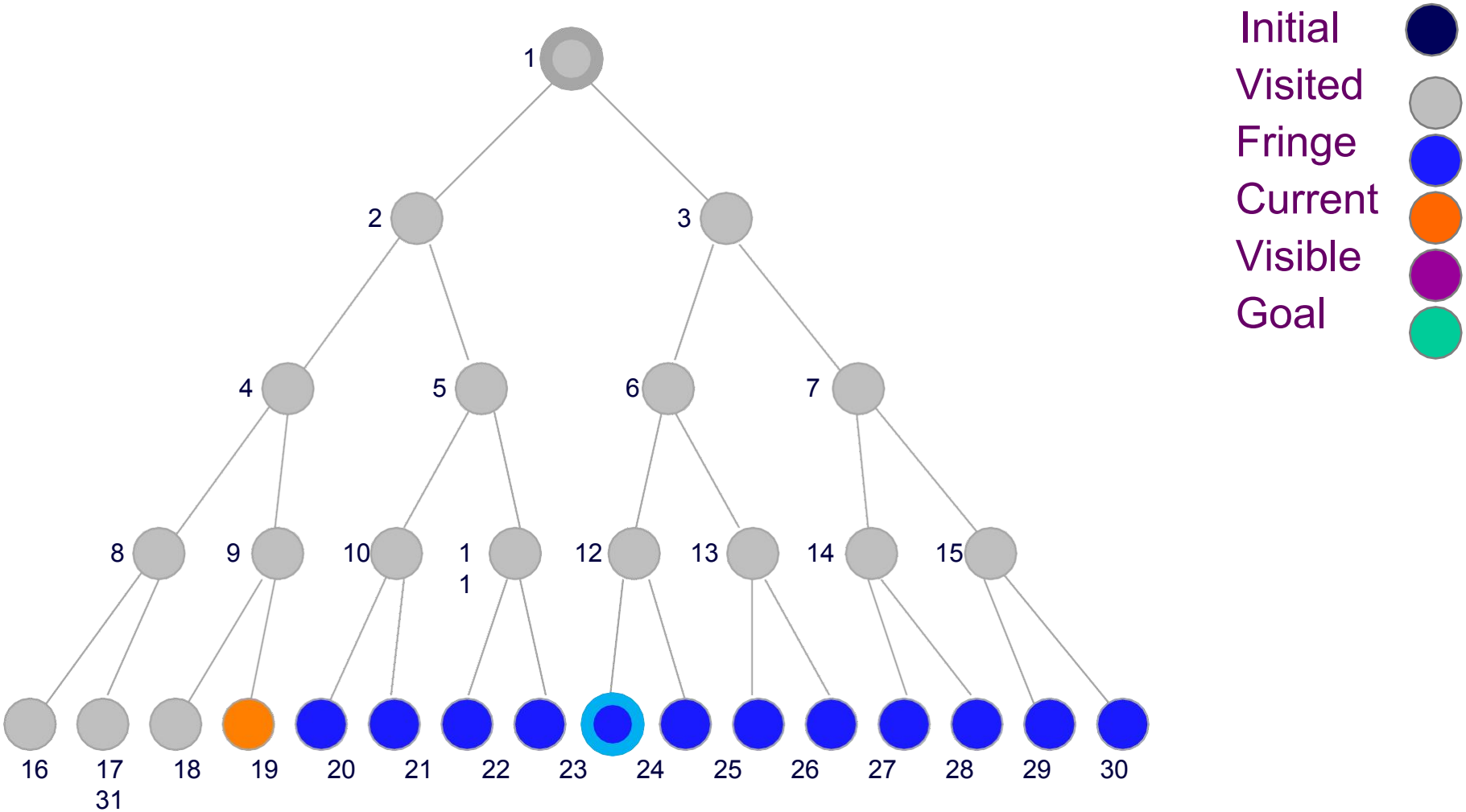
Fringe:
[18,19,20,21,22,23,24,25,26,27,28,29,30,31]

Breadth-First Snapshot 18



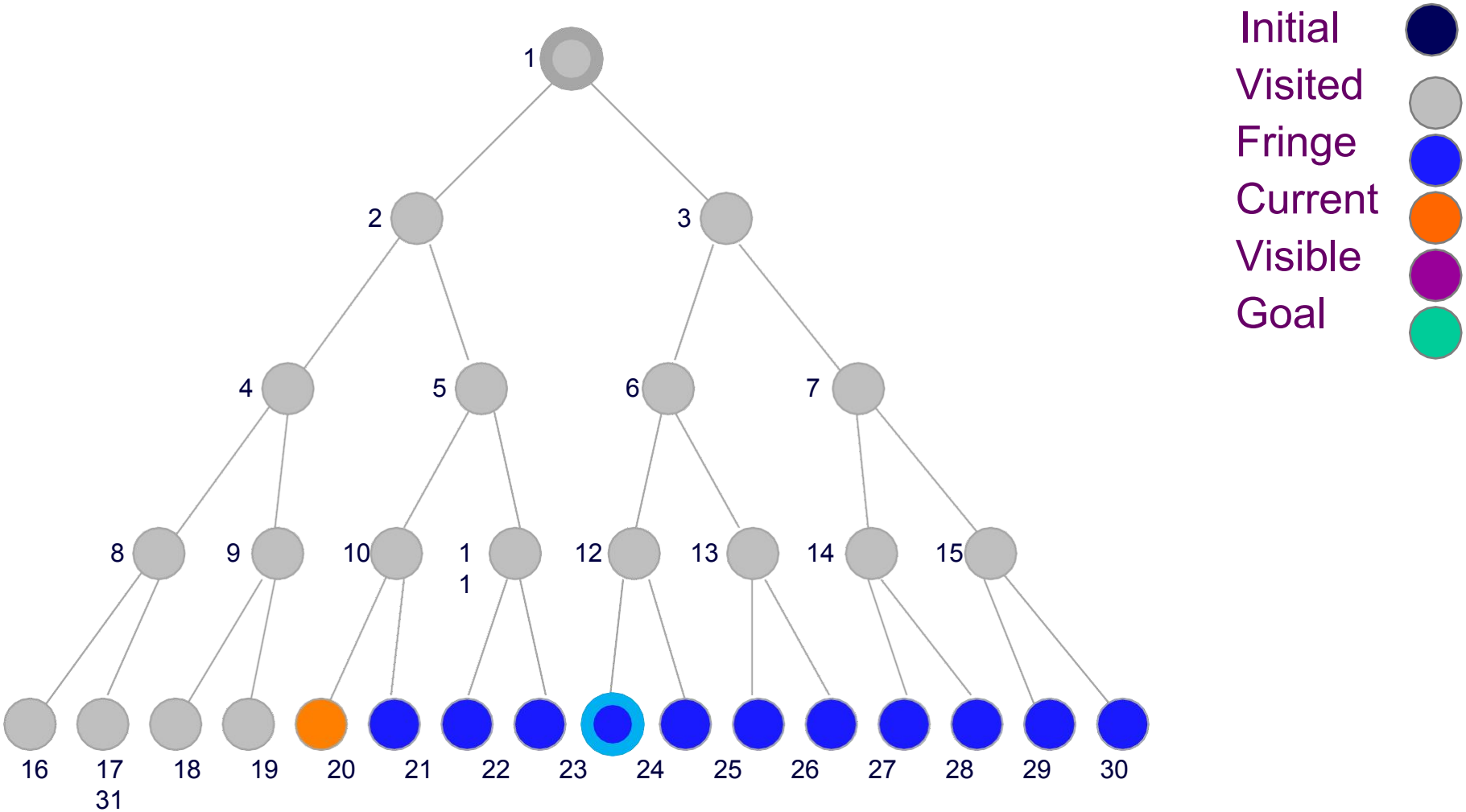
Fringe:
[19,20,21,22,23,24,25,26,27,28,29,30,31]

Breadth-First Snapshot 19



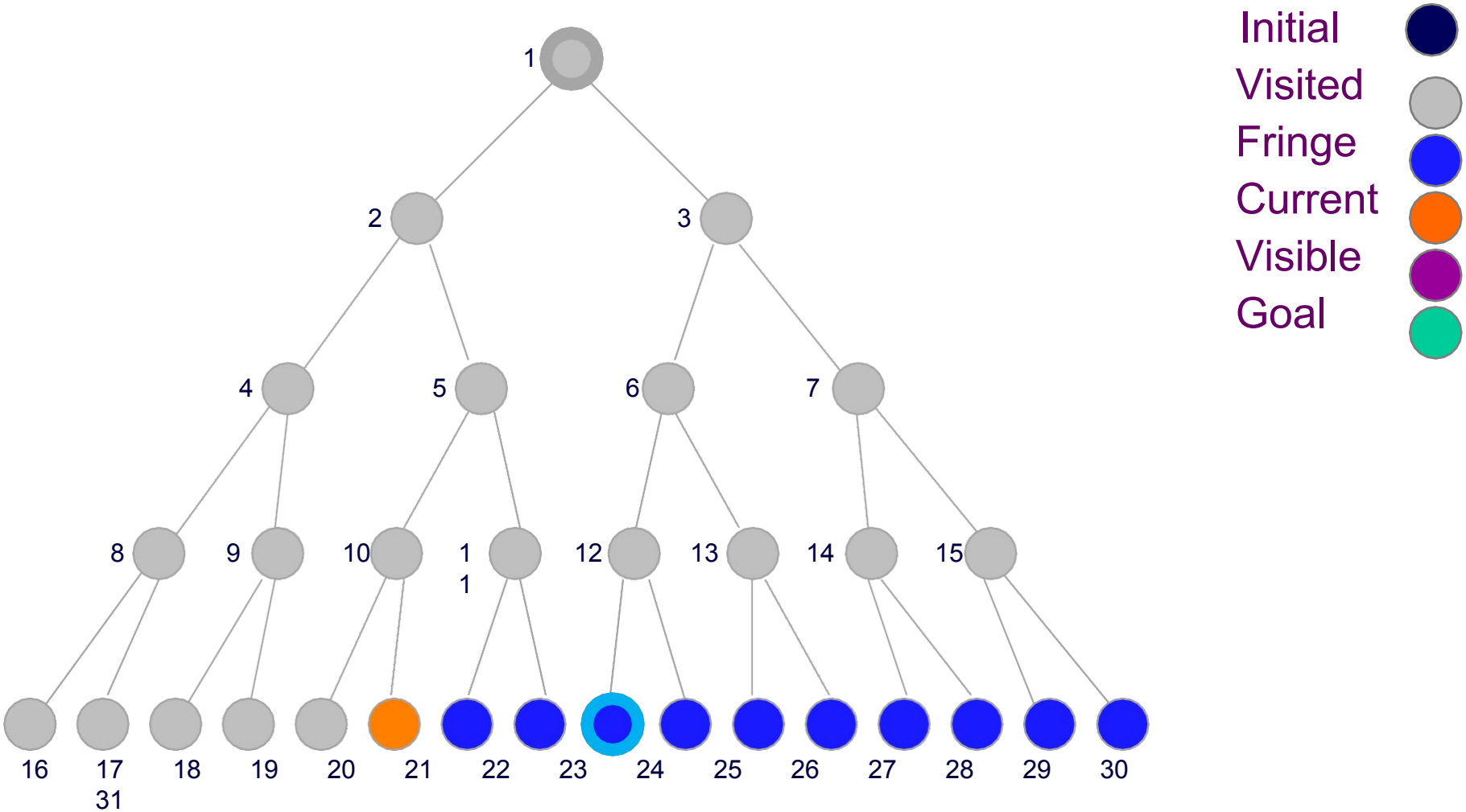
Fringe:
[20,21,22,23,24,25,26,27,28,29,30,31]

Breadth-First Snapshot 20



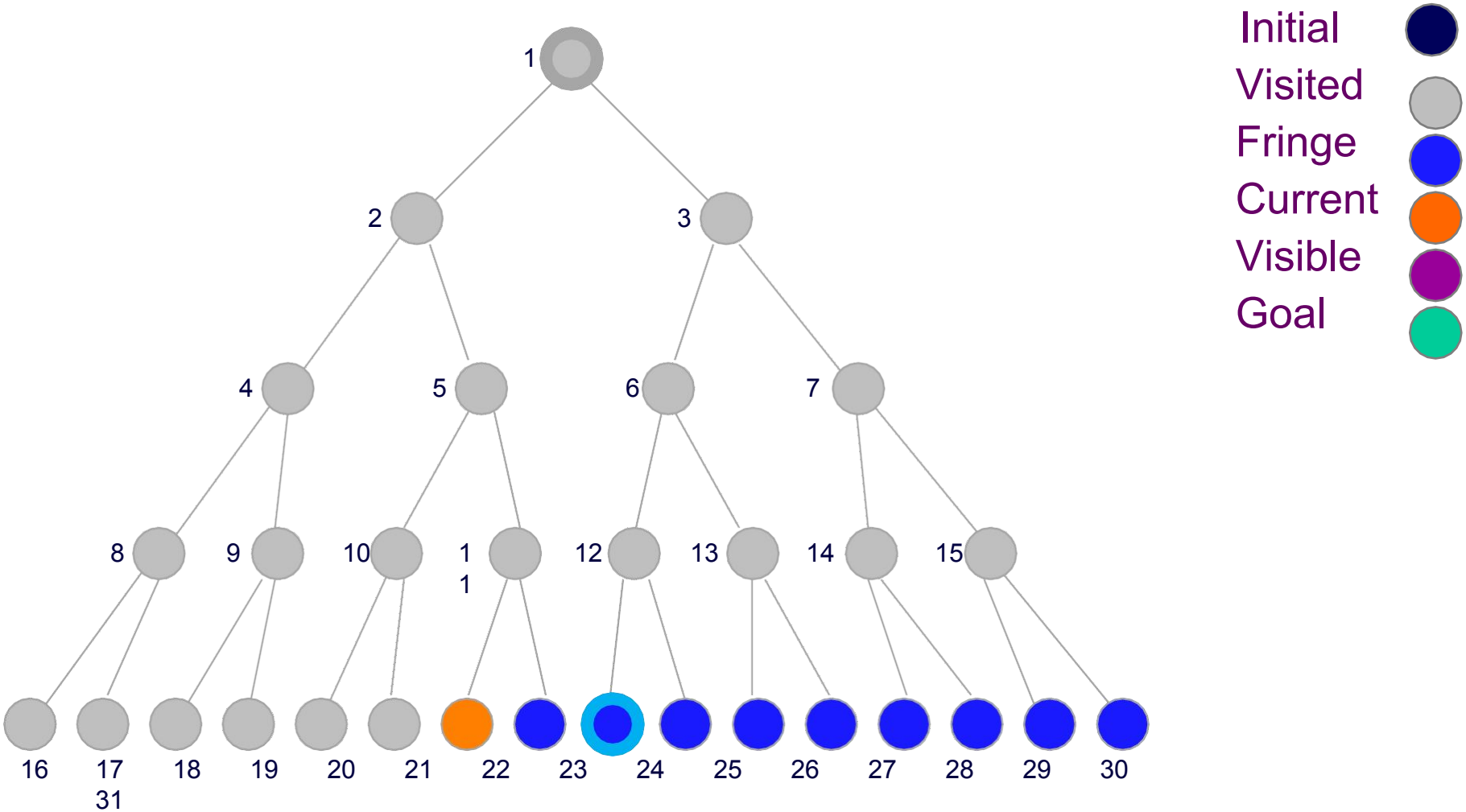
Fringe:
[21,22,23,24,25,26,27,28,29,30,31]

Breadth-First Snapshot 21



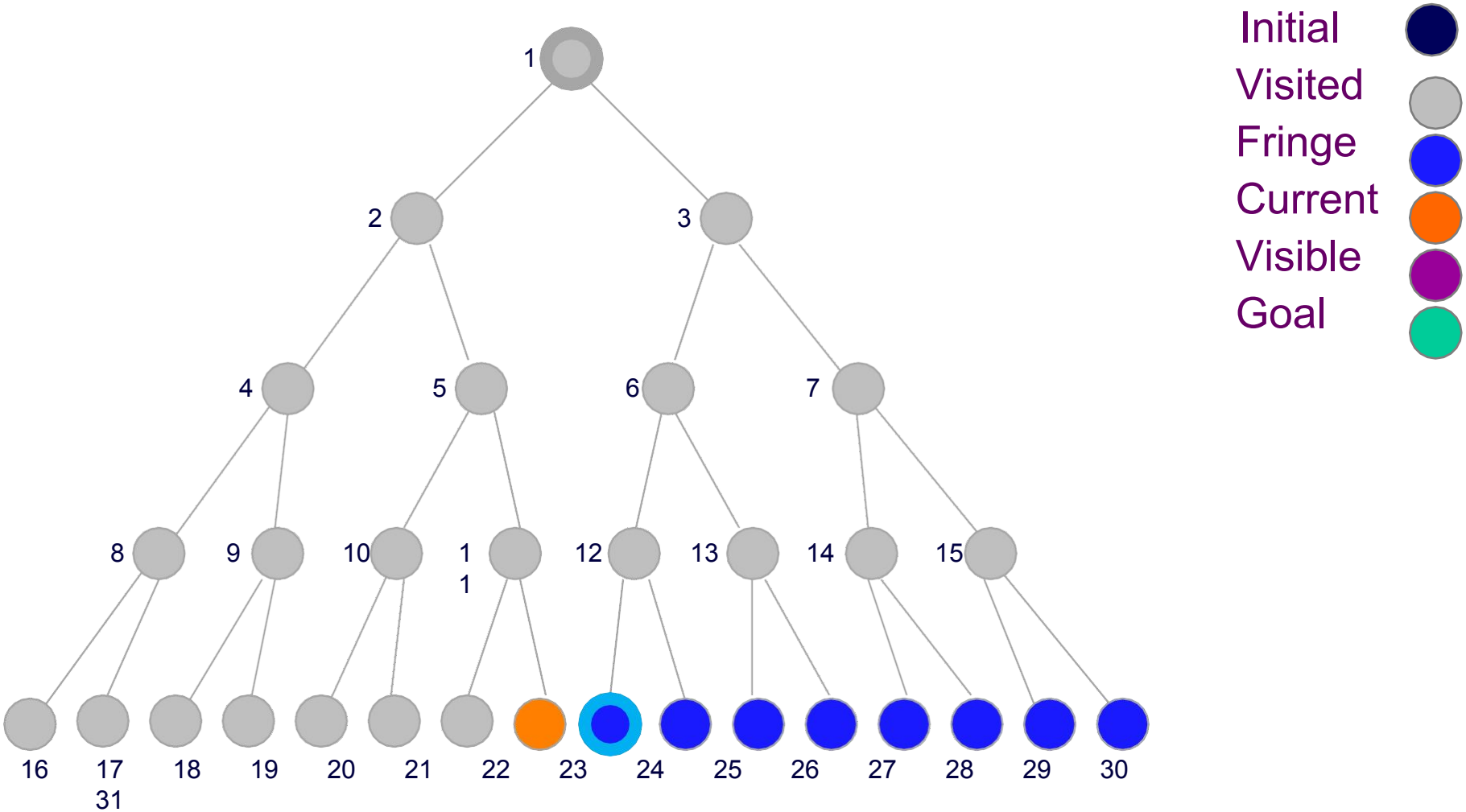
Fringe:
[22,23,24,25,26,27,28,29,30,31]

Breadth-First Snapshot 22



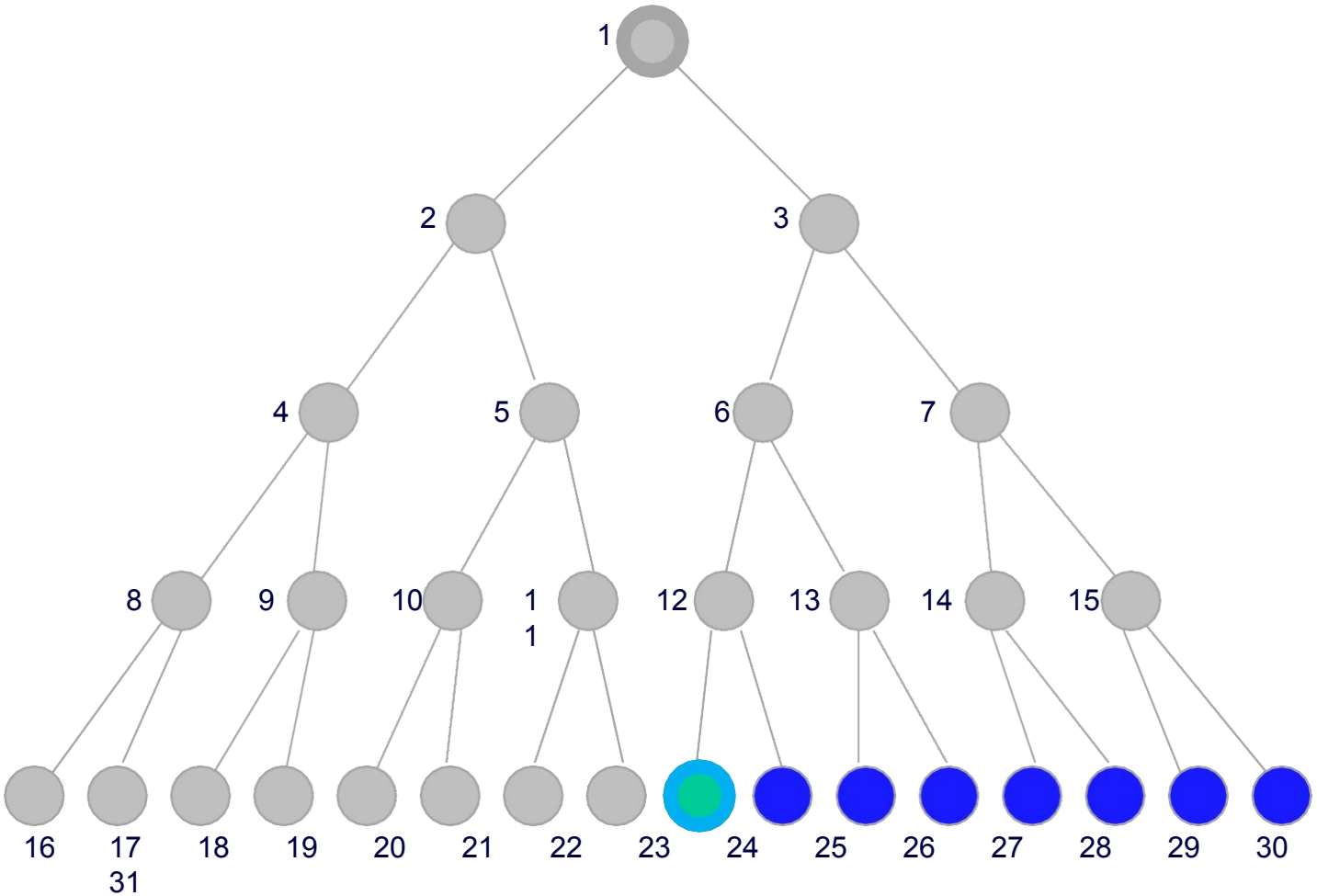
Fringe:
[23,24,25,26,27,28,29,30,31]

Breadth-First Snapshot 23



Fringe: [24,25,26,27,28,29,30,31]

Breadth-First Snapshot 24



Initial
Visited
Fringe
Current
Visible
Goal

Note:
The goal test is positive for this node, and a solution is found in 24 steps.

Fringe: [25,26,27,28,29,30,31]

Properties of Breadth-First Search (BFS)

Completeness: Yes (if *b* is finite), a solution will be found if exists.

Time Complexity: (nodes until the solution) Optimality: Yes

Criterion	Breadth-First
Complete?	Yes
Time	$O(b^{d+1})$
Space	$O(b^{d+1})$
Optimal?	Yes

b Branching Factor

d The depth of the goal

Suppose the branching factor b=3, and the goal is at depth d=20:

– Then we need 3^{20} time to finish.

Not suitable for searching large graphs

2- Uniform-Cost -First

Uniform-Cost -First

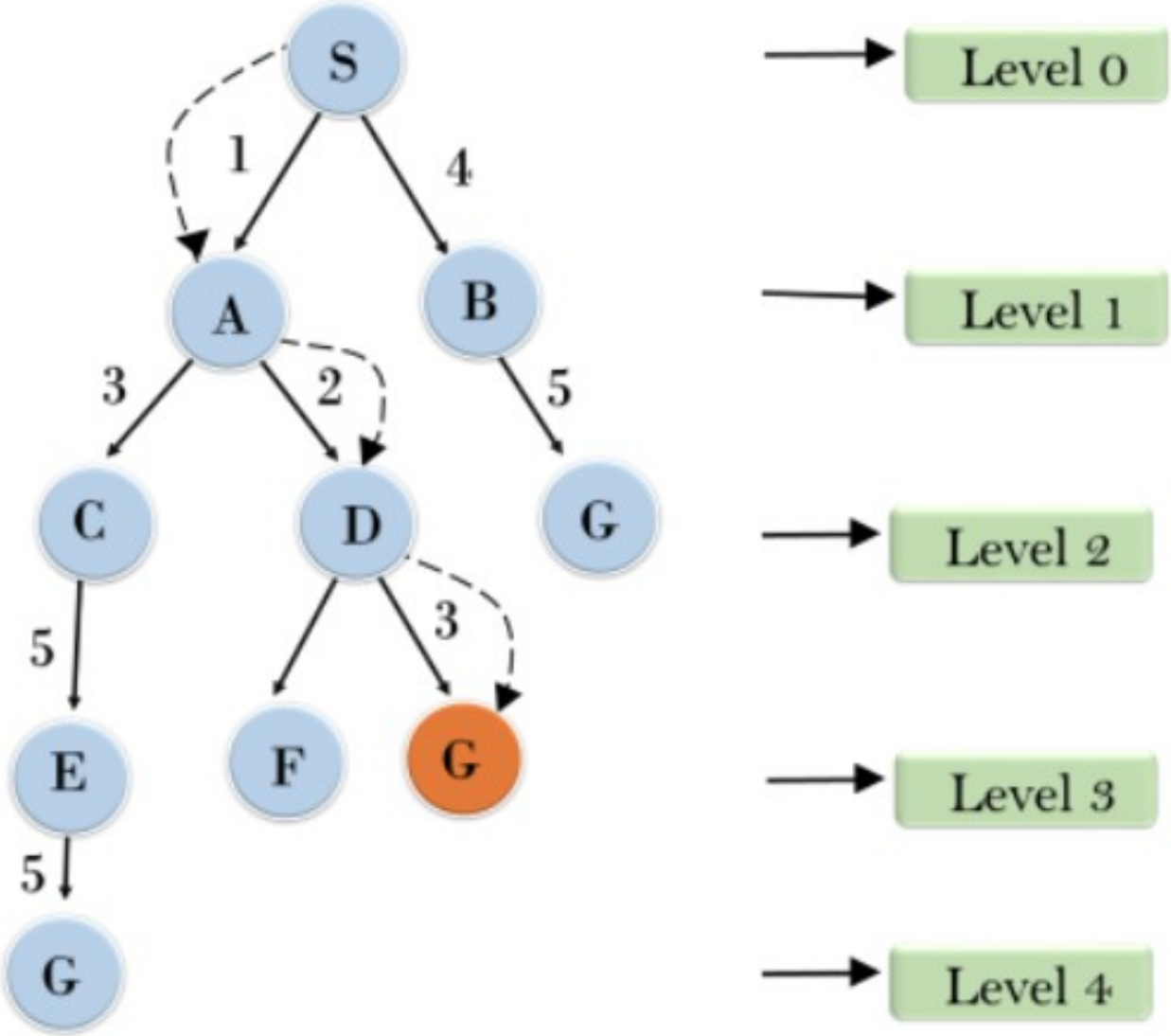
- The primary goal of the uniform-cost search is to find a path to the goal node which has the lowest cumulative cost.

  Uniform-cost search expands nodes according to their path costs form the root node.

  It can be used to solve any graph/tree where the optimal cost is in demand.

  Uniform cost search is equivalent to BFS algorithm if the path cost of all edges is the same.

Uniform Cost Search

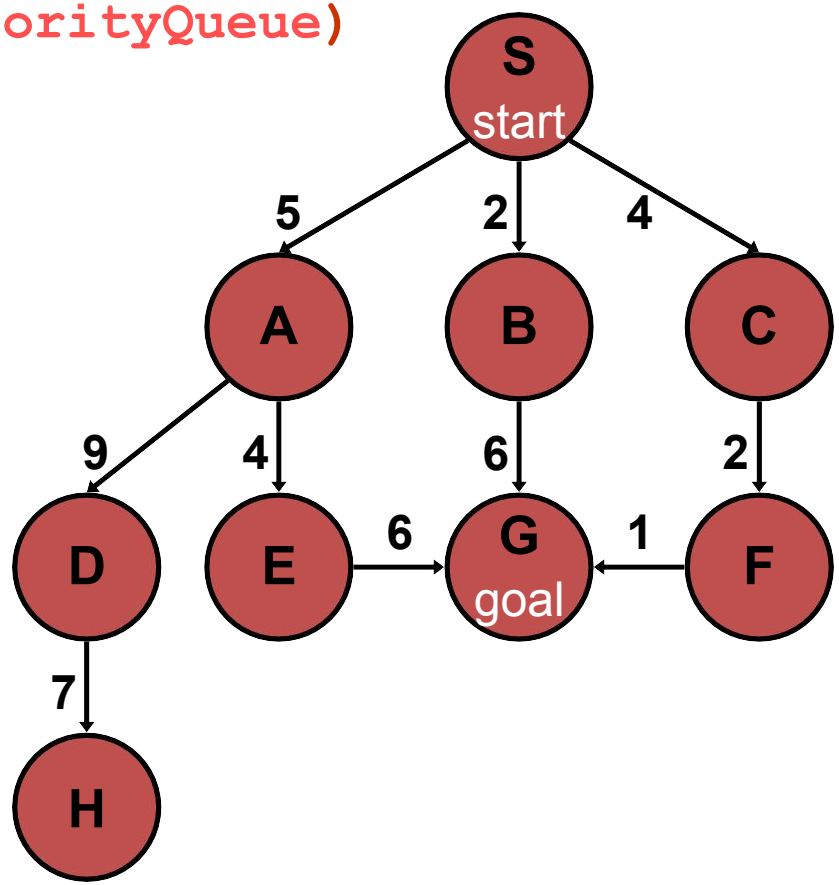


Uniform-Cost Search (UCS)

```
generalSearch(problem, priorityQueue)
```

# of nodes tested: 0, expanded: 0

Closed Node	Open Node
	{S}

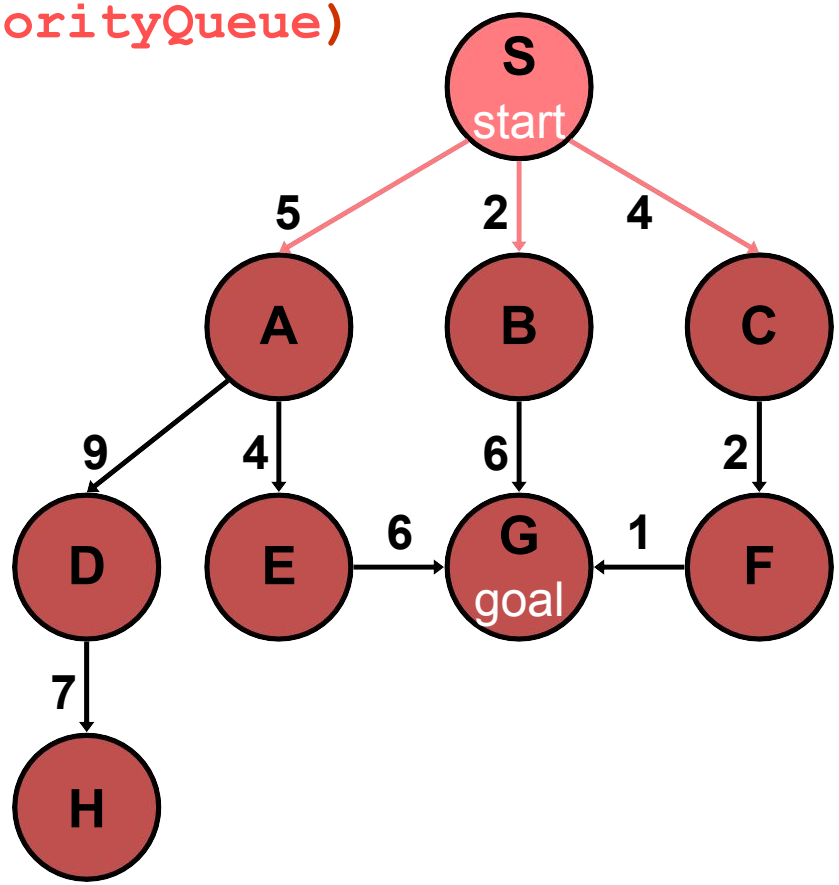


Uniform-Cost Search (UCS)

```
generalSearch(problem, priorityQueue)
```

# of nodes tested: 1, expanded: 1

Closed Node	Open Node
	{S:0}
S not goal	{B:2,C:4,A:5}

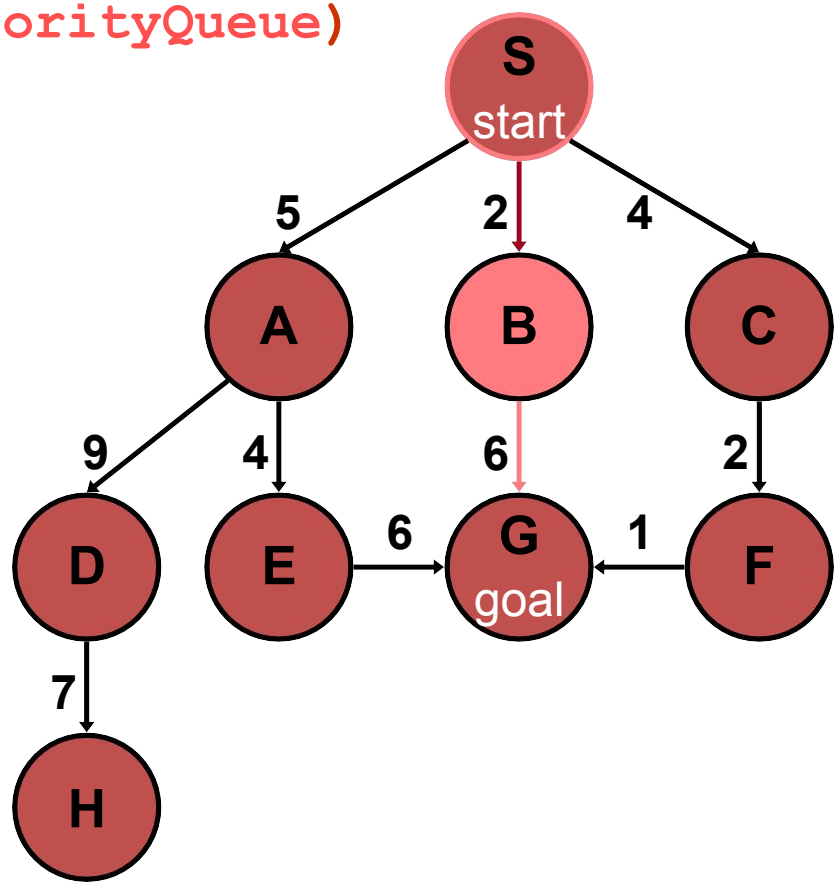


Uniform-Cost Search (UCS)

```
generalSearch(problem, priorityQueue)
```

# of nodes tested: 2, expanded: 2

Closed Node	Open Node
	{S}
S	{B:2,C:4,A:5}
B not goal	{C:4,A:5,G:2+6}

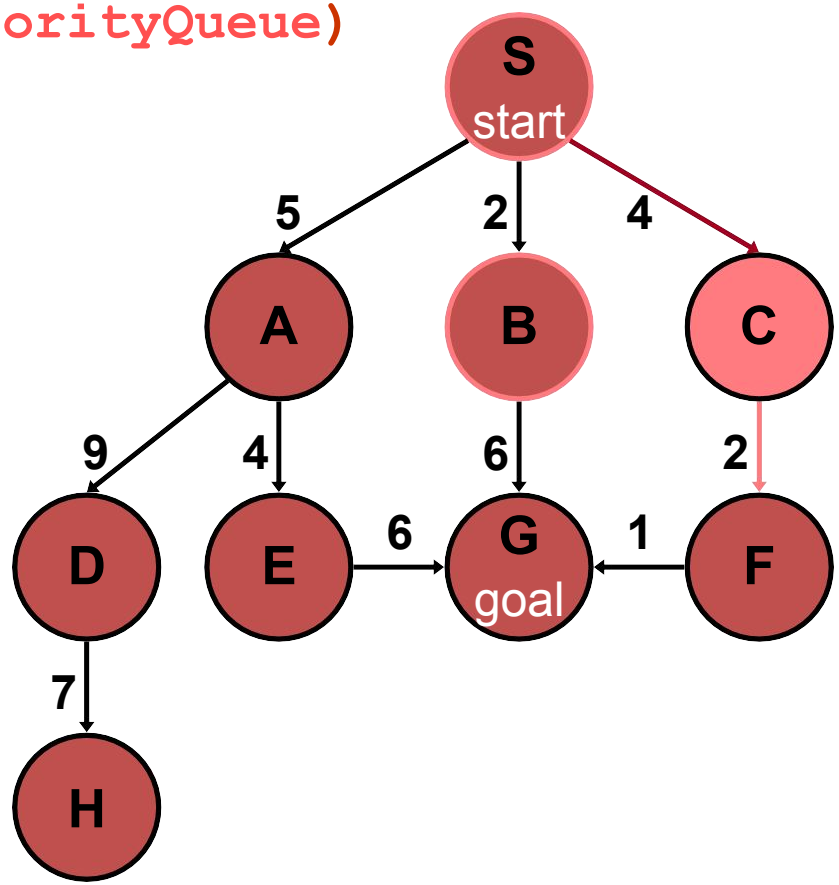


Uniform-Cost Search (UCS)

```
generalSearch(problem, priorityQueue)
```

# of nodes tested: 3, expanded: 3

Closed Node	Open Node
	{S}
S	{B:2,C:4,A:5}
B	{C:4,A:5,G:8}
C not goal	{A:5,F:4+2,G:8}

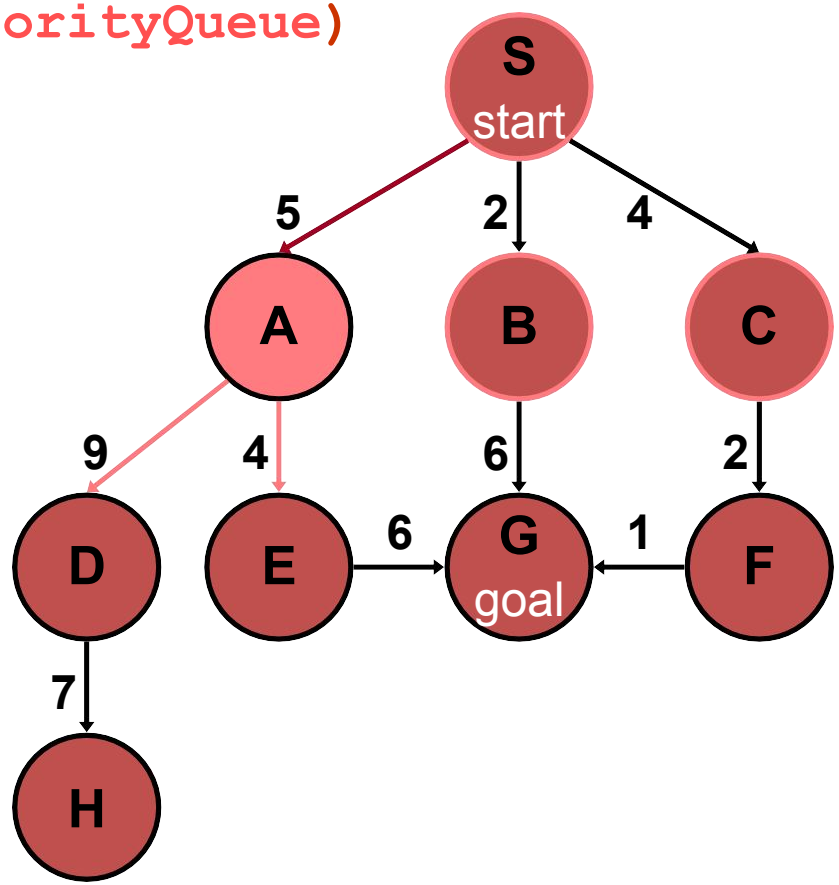


Uniform-Cost Search (UCS)

`generalSearch(problem, priorityQueue)`

# of nodes tested: 4, expanded: 4

Closed Node	Open Node
	{S}
S	{B:2,C:4,A:5}
B	{C:4,A:5,G:8}
C	{A:5,F:6,G:8}
A not goal	{F:6,G:8,E:5+4, D:5+9}

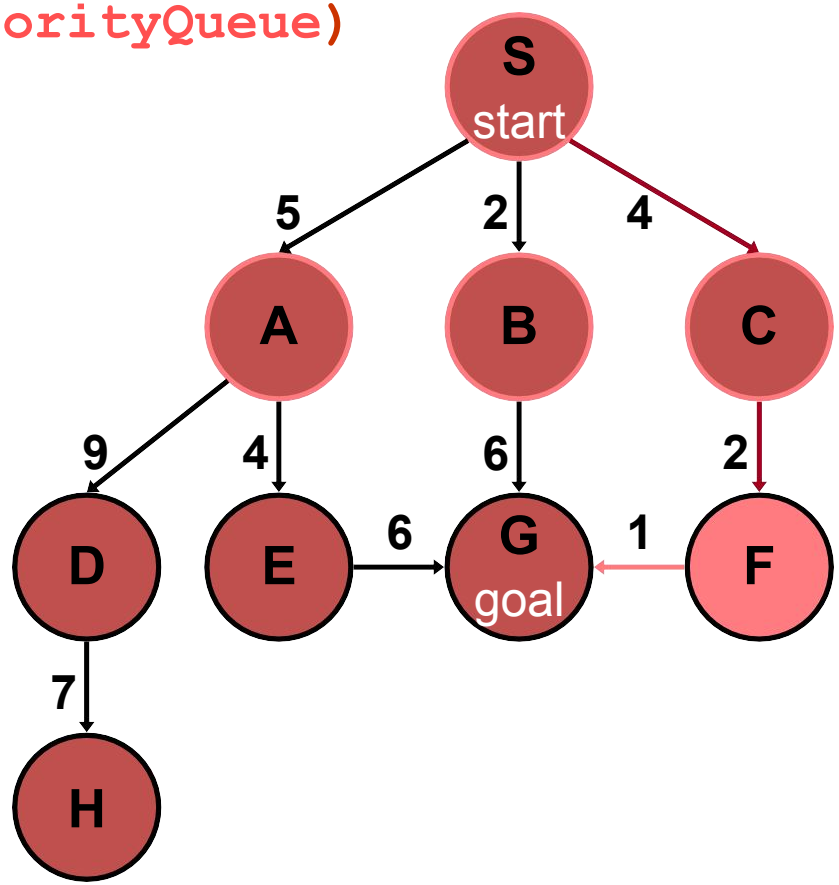


Uniform-Cost Search (UCS)

```
generalSearch(problem, priorityQueue)
```

# of nodes tested: 5, expanded: 5

Closed Node	Open Node
	{S}
S	{B:2,C:4,A:5}
B	{C:4,A:5,G:8}
C	{A:5,F:6,G:8}
A	{F:6,G:8,E:9,D:14}
F not goal	{G:4+2+1,G:8,E:9,D:14}

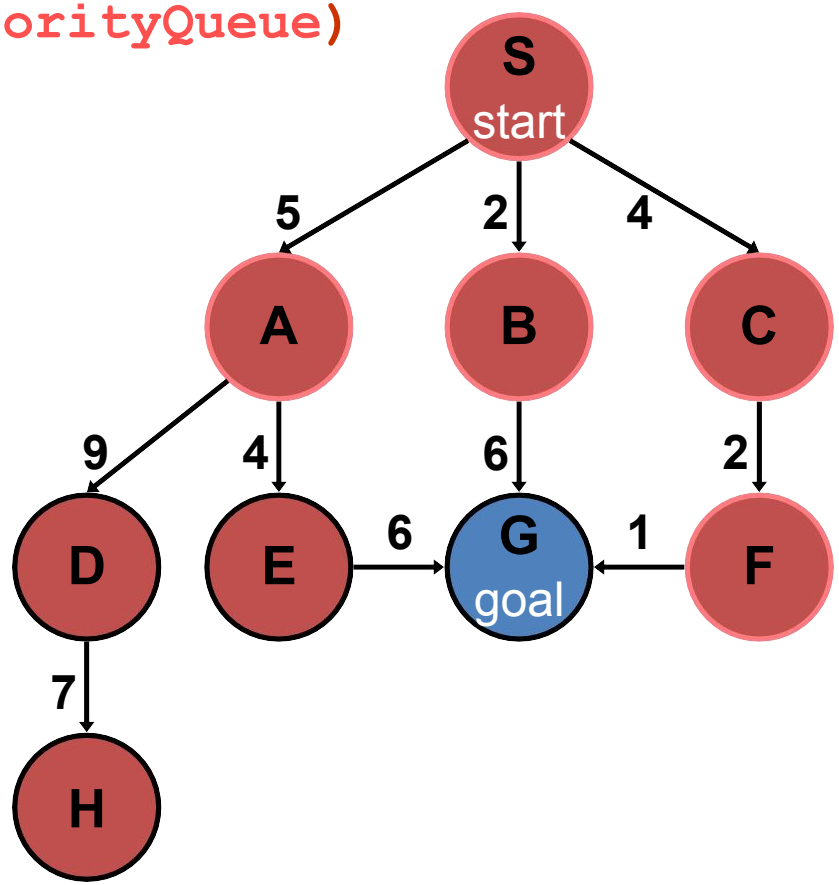


Uniform-Cost Search (UCS)

```
generalSearch(problem, priorityQueue)
```

# of nodes tested: 6, expanded: 5

Closed Node	Open Node
	{S}
S	{B:2,C:4,A:5}
B	{C:4,A:5,G:8}
C	{A:5,F:6,G:8}
A	{F:6,G:8,E:9,D:14}
F	{G:7,G:8,E:9,D:14}
G goal	{G:8,E:9,D:14} no expand

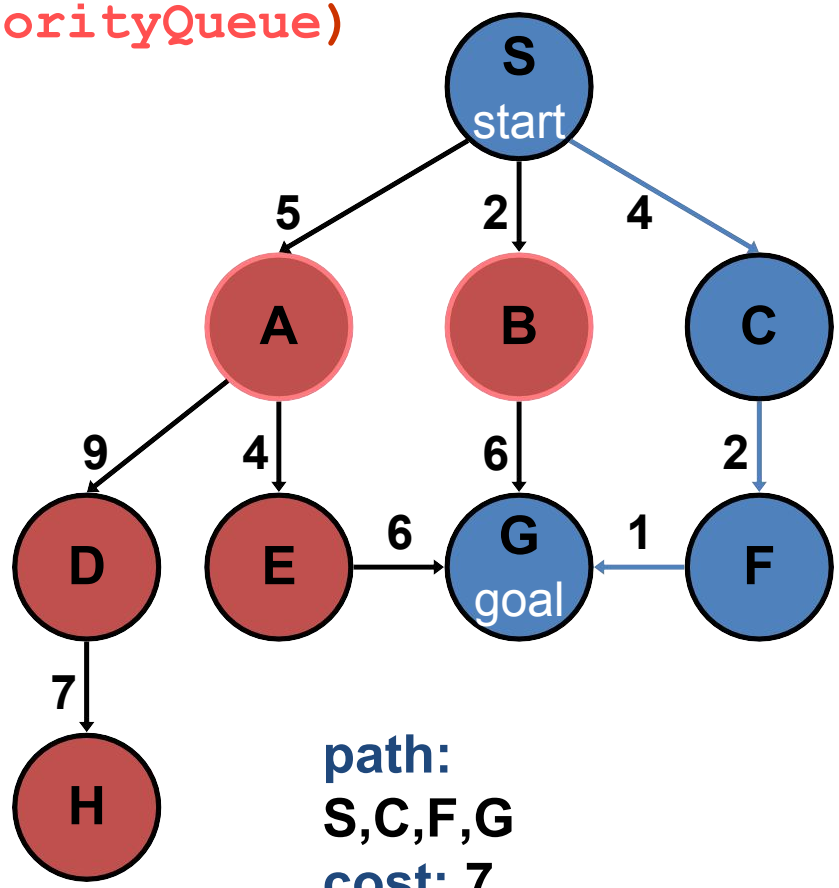


Uniform-Cost Search (UCS)

```
generalSearch(problem, priorityQueue)
```

# of nodes tested: 6, expanded: 5

Closed Node	Open Node
	{S}
S	{B:2,C:4,A:5}
B	{C:4,A:5,G:8}
C	{A:5,F:6,G:8}
A	{F:6,G:8,E:9,D:14}
F	{G:7,G:8,E:9,D:14}
G	{G:8,E:9,D:14}



path:
S,C,F,G
cost: 7

Properties of Uniform-cost Search (UCS)

Completeness Yes (if *b* is finite, and step cost is positive)

Time Complexity much larger than b^d and just b^d if all steps have the same cost.

Criterion	Uniform-Cost
Complete?	Yes
Time	$O(b^{\lceil C^*/\epsilon \rceil})$
Space	$O(b^{\lceil C^*/\epsilon \rceil})$
Optimal?	Yes

Space Complexity: as above

Optimality:

Yes

Requires that the goal test being applied when a node is removed from the nodes list rather than when the node is first generated while its parent node is expanded.

b	Branching Factor
d	Depth of the goal/tree

Breadth-First vs. Uniform-Cost

Breadth-first search (BFS) is a special case of uniform-cost search when all edge costs are positive and identical.

Breadth-first always expands the shallowest node

- Only optimal if all step-costs are equal

Uniform-cost considers the overall path cost

- Optimal for any (reasonable) cost function
  - non-zero, positive
- Gets stuck down in trees with many fruitless, short branches
  - low path cost, but no goal node

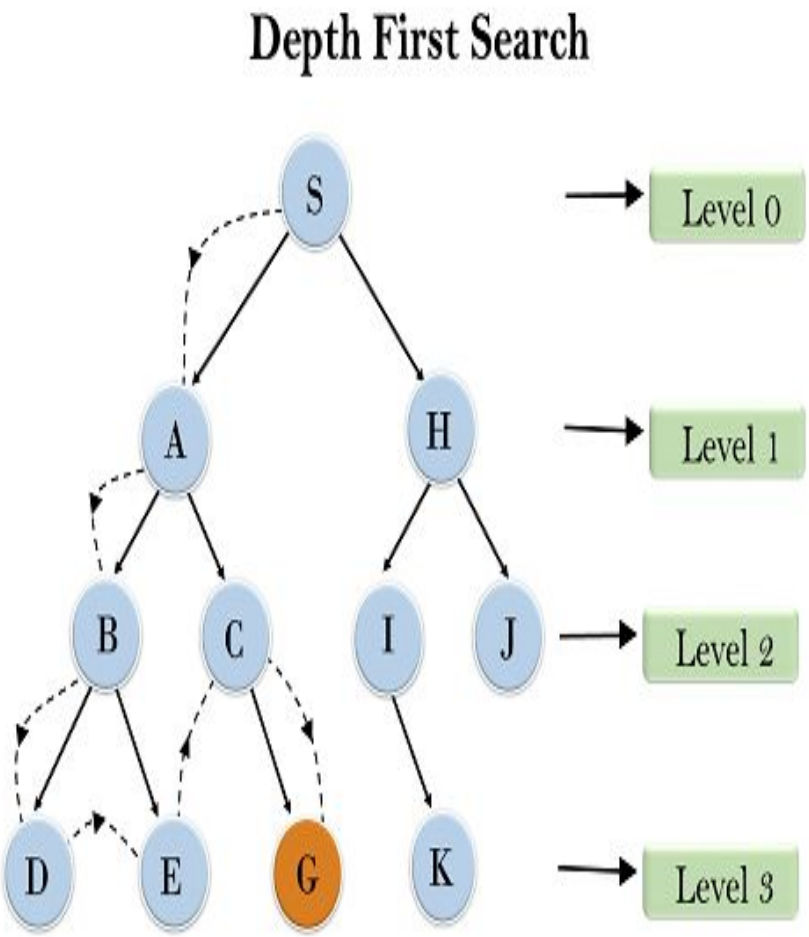
Both are complete for non-extreme problems

- Finite number of branches
- Strictly positive search function

3- Depth-First Search

Depth-First Search

Based on [4]



It will start searching from root node S, and traverse A, then B, then D and E, after traversing E, it will backtrack the tree as E has no other successor and still goal node is not found.

After backtracking it will traverse node C and then G, and here it will terminate as it found goal node.

Properties of Depth-First Search

Complete: No: fails in infinite-depth spaces, spaces with loops
– Yes, complete in finite spaces

Time: $O(b^m)$: terrible if m is much larger than d

– but if solutions are dense, may be much faster than breadth-first

Space: $O(bm)$

Optimal: No

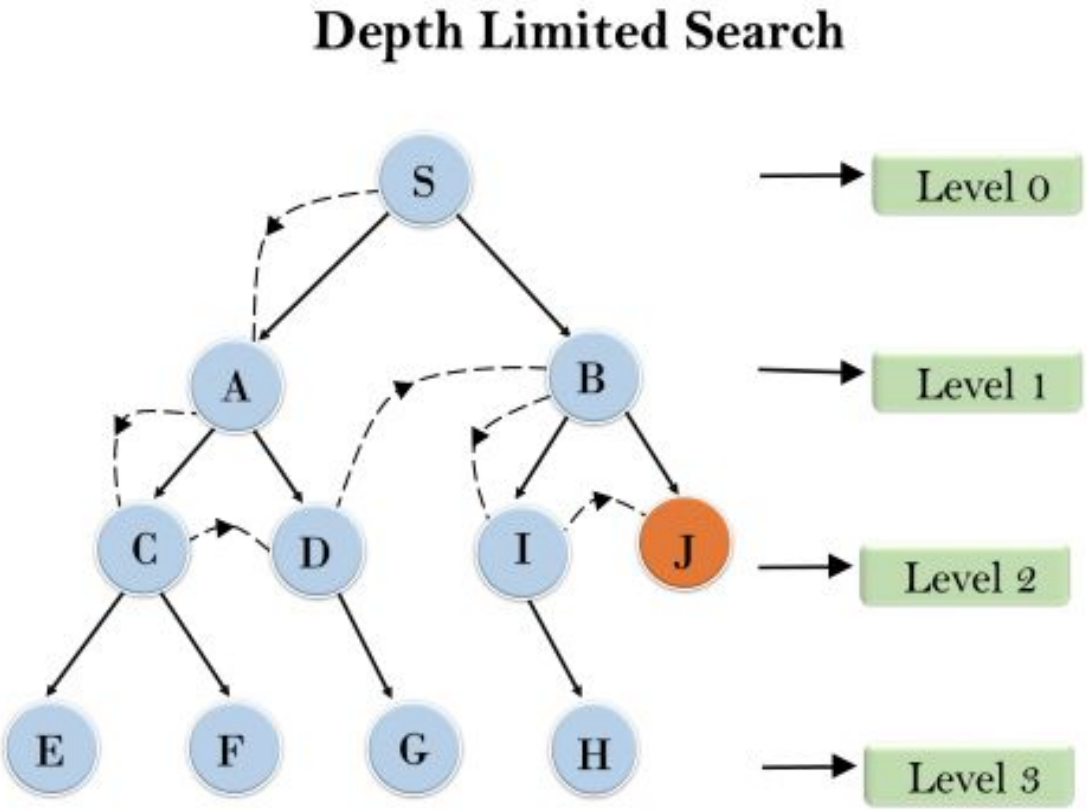
Criterion	Depth-First
Complete?	No
Time	$O(b^m)$
Space	$O(bm)$
Optimal?	No

4- Depth-Limited Search

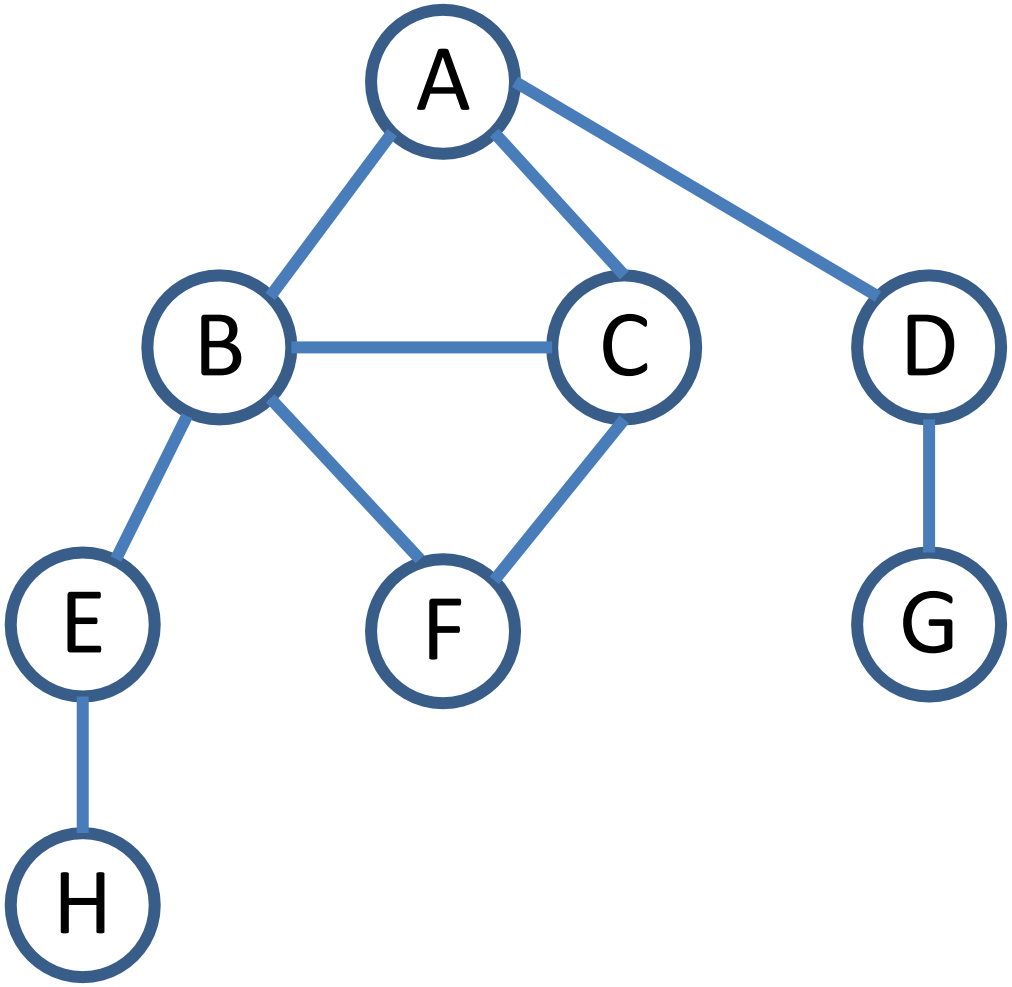
4- Depth-Limited Search

A depth-limited search algorithm is similar to depth-first search with a predetermined limit.
Depth-limited search can solve the drawback of the infinite path in the Depth-first search.
In this algorithm, the node at the depth limit will treat as it has no successor nodes further.
Depth-limited search can be terminated with two Conditions of failure:
Standard failure value: It indicates that problem does not have any solution.
Cutoff failure value: It defines no solution for the problem within a given depth limit.

Example


Depth Limited Search
S
A
B
C
D
I
J
E
F
G
H
Level 0
Level 1
Level 2
Level 3

4- Example DLS



Depth = 2

ABCFEDG

4- Depth-Limited Search

Similar to depth-first, but with a limit

- i.e., nodes at depth *l* have no successors
- Overcomes problems with infinite paths
- Sometimes a depth limit can be inferred or estimated from the problem description
  - In other cases, a good depth limit is only known when the problem is solved
- must keep track of the depth

- Complete? no (if goal beyond l (l<d), or infinite branch length)
- Time? $O(b^{l})$
- Space? $O(bl)$
- Optimal? No (if l < d)

b branching factor l depth limit

Criterion	Depth-Limited
Complete?	No
Time	$O(b^l)$
Space	$O(bl)$
Optimal?	No

5- Iterative Deepening Depth-First Search

5- Iterative Deepening Depth-First Search

The iterative deepening algorithm is a combination of DFS and BFS algorithms.

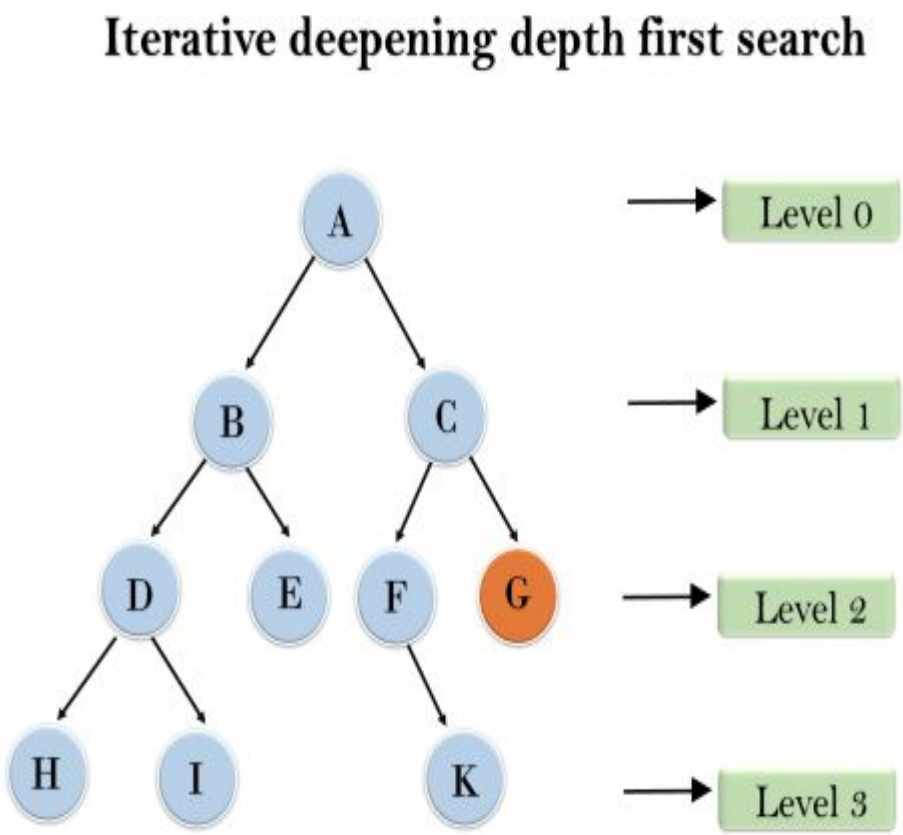
This search algorithm finds out the best depth limit and does it by gradually increasing the limit until a goal is found.

This algorithm performs depth-first search up to a certain "depth limit", and it keeps increasing the depth limit after each iteration until the goal node is found.

This Search algorithm combines the benefits of Breadth-first search's fast search and depth-first search's memory efficiency.

The iterative search algorithm is useful uninformed search when search space is large, and depth of goal node is unknown.

5- Iterative Deepening Depth-First Search



1'st Iteration-----> A

2'nd Iteration----> A, B, C

3'rd Iteration------>A, B, D, E, C, F, G

4'th Iteration------>A, B, D, H, I, E, C, F, K, G

In the fourth iteration, the algorithm will find the goal node.

Iterative Deepening Depth-First Search

Applies LIMITED-DEPTH with increasing depth limits

- Combines advantages of BREADTH-FIRST and DEPTH- FIRST
- It searches to depth 0 (root only), then if that fails it searches to depth 1, then depth 2, etc.

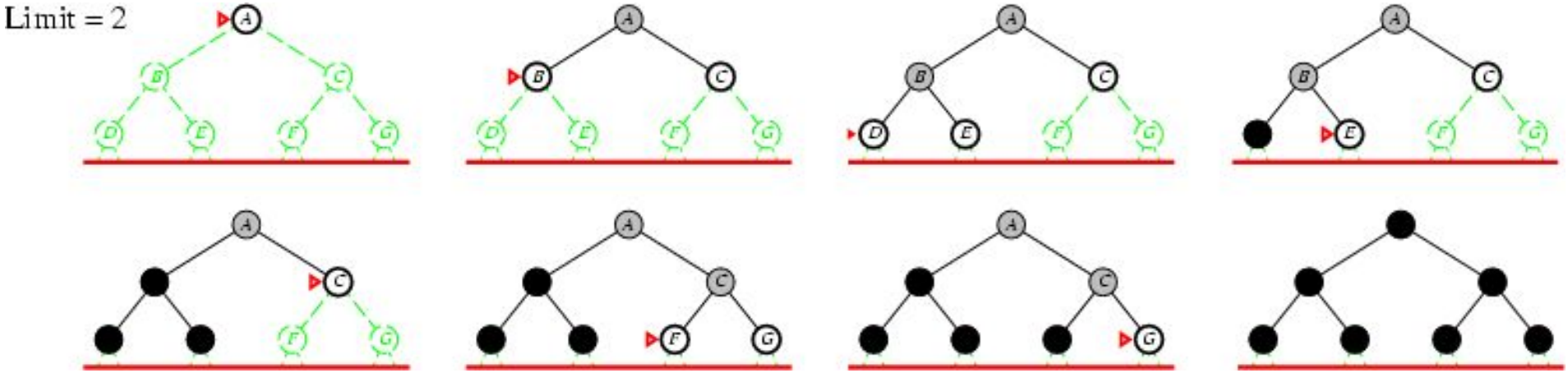
```
function ITERATIVE-DEEPENING-SEARCH( problem) returns a solution, or failure
    inputs: problem, a problem
    for depth ← 0 to ∞ do
        result ← DEPTH-LIMITED-SEARCH( problem, depth)
        if result ≠ cutoff then return result
```

Iterative deepening search l =0

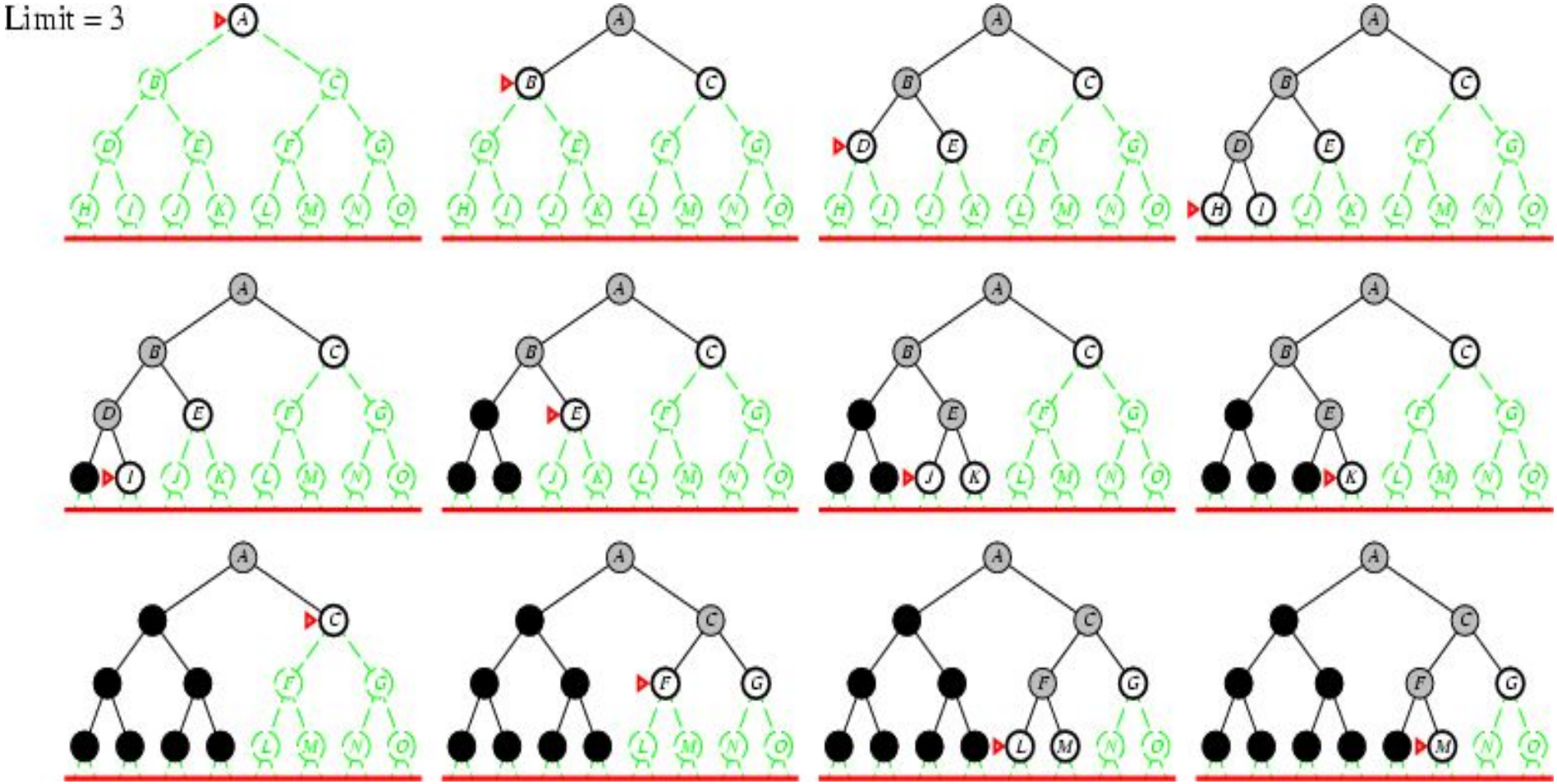
Limit = 0

Iterative deepening search l =1

Iterative deepening search l =2



Iterative deepening search l =3



Properties of Iterative Deepening Search

Complete: Yes (if the b is finite)

Time: $(d+1)b^0 + d\,b^1 + (d-1)b^2 + \ldots + b^d = O(b^d)$

Space: $O(bd)$

Optimal: Yes, if step cost = 1

Criterion	Iterative Deepening
Complete?	Yes
Time	$O(b^d)$
Space	$O(bd)$
Optimal?	Yes

b branching factor d Tree/goal depth

6- Bi-directional Search

6- Bi-directional Search

Bidirectional search algorithm runs two simultaneous searches, one form initial state called as forward-search and other from goal node called as backward-search, to find the goal node.

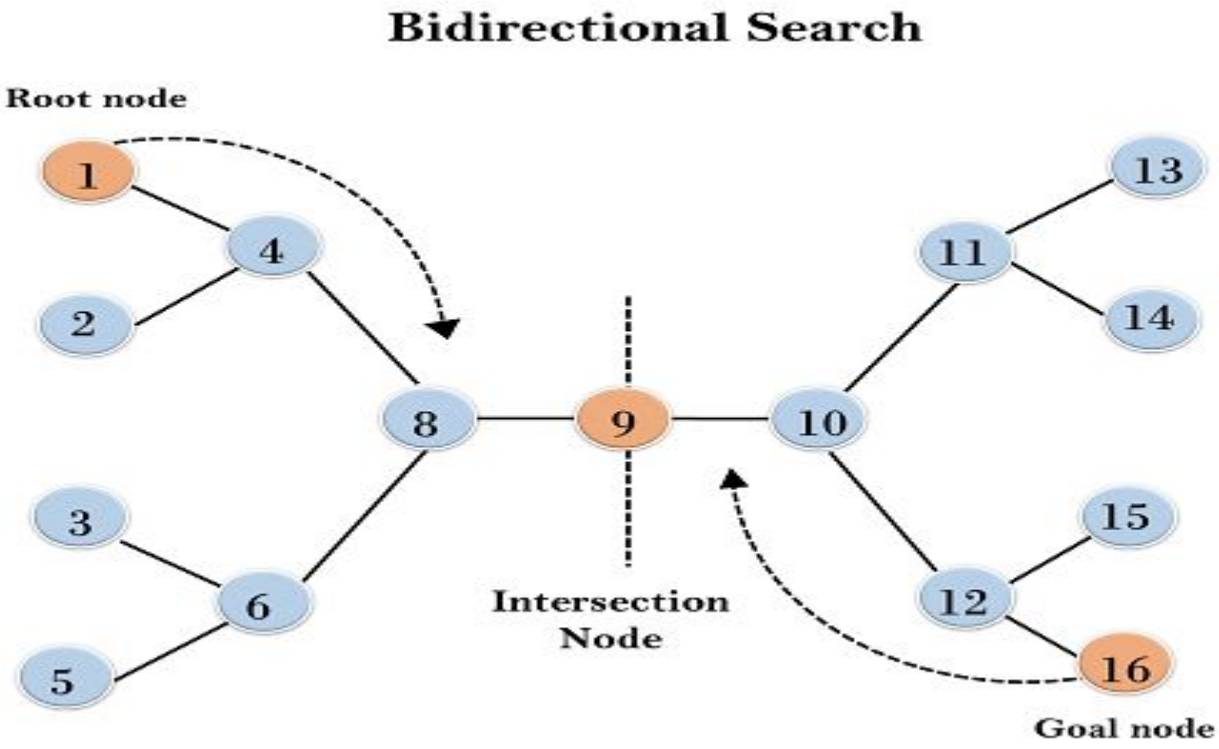
Bidirectional search replaces one single search graph with two small subgraphs in which one starts the search from an initial vertex and other starts from goal vertex.
The search stops when these two graphs intersect each other.

Bidirectional search can use search techniques such as BFS, DFS, etc.

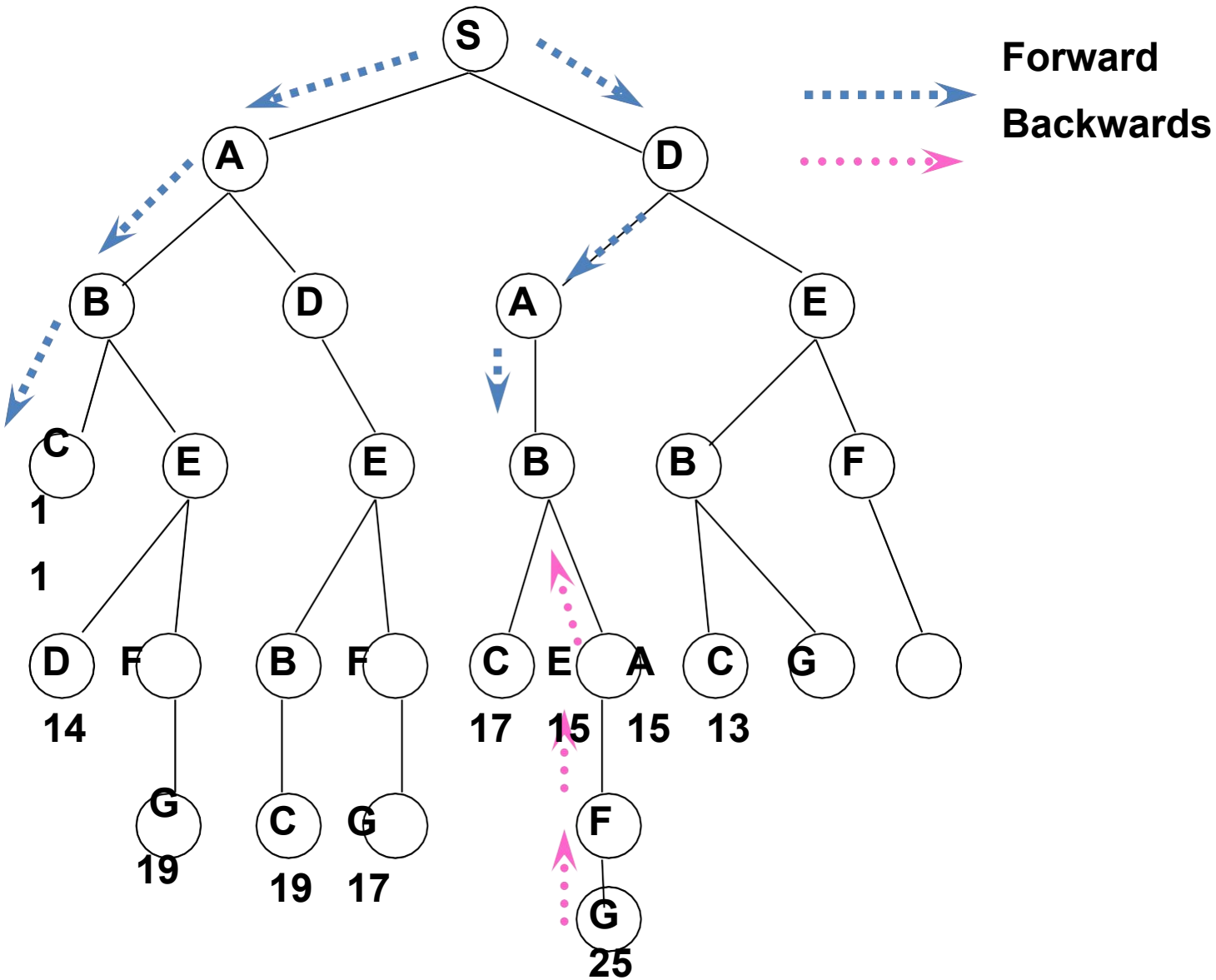
6- Bi-directional Search

- Both search forward from initial state, and backwards from goal.
- Stop when the two searches meet in the middle.
- Motivation: $b^{d/2} + b^{d/2}$ is much less than b^d
- Implementation
  - Replace the goal test with a check to see whether the frontiers of the two searches intersect, if yes solution is found

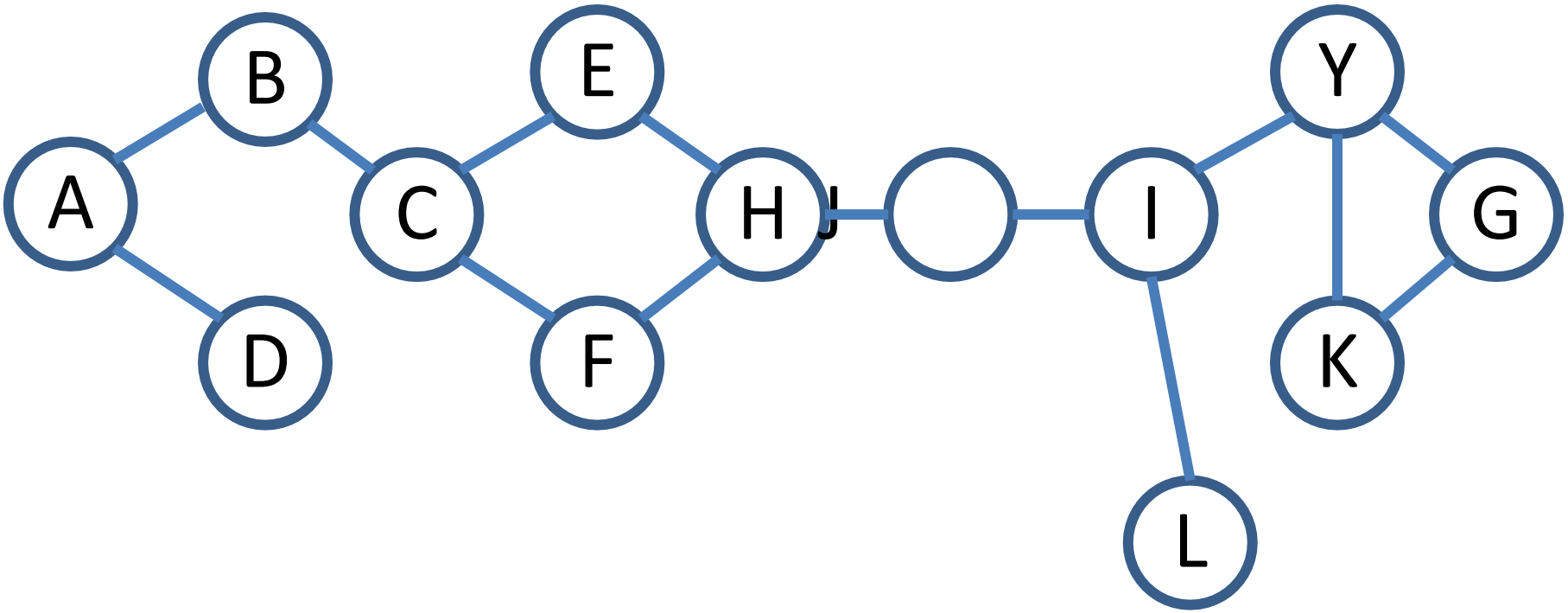
Bi-directional Search Example



6- Bi-directional Search



6- Bi-directional Search Example Using BFS



6- Bi-directional Search

Completeness: Bidirectional Search is complete if we use BFS in both searches.
Time Complexity: Time complexity of bidirectional search using BFS is $\mathbf{O(b^d)}$.
Space Complexity: Space complexity of bidirectional search is $\mathbf{O(b^d)}$.
Optimal: Bidirectional search is Optimal.

Summary (When to use what)

Breadth-First Search:

- Some solutions are known to be shallow

Uniform-Cost Search:

- Actions have varying costs
- Least cost solution is the required

This is the only uninformed search that worries about costs.

Depth-First Search:

- Many solutions exist
- Know (or have a good estimate of) the depth of solution

Iterative-Deepening Search:

- Space is limited and the shortest solution path is required

Summary

- Problem formulation

- Variety of uninformed search strategies

Criterion	Breadth-First	Uniform-Cost	Depth-First	Depth-Limited	Iterative Deepening
Complete?	Yes	Yes	No	No	Yes
Time	$O(b^{d+1})$	$O(b^{\lceil C^*/\epsilon \rceil})$	$O(b^m)$	$O(b^l)$	$O(b^d)$
Space	$O(b^{d+1})$	$O(b^{\lceil C^*/\epsilon \rceil})$	$O(bm)$	$O(bl)$	$O(bd)$
Optimal?	Yes	Yes	No	No	Yes